Universidad Europea

Introduction to Computer Vision

Lapland University of Applied Sciences

Dr. Manuel García Fernández
manuel.garcia2@universidadeuropea.es

Ve más allá

- Convolutional Neural Networks **01**
- Deep Learning **02**
- Transfer Learning **03**
- Image Segmentation **04**

01

Convolutional Neural Networks

Convolutional Neural Networks

We saw in the previous topic that, using the mathematical concept of Convolution, we can establish a similarity between an image and a predefined shape (called a Kernel) described in the convolution integral.

For simple shapes (such as squares) it is trivial to define the kernel. Even for complex shapes, as long as we know the analytical form the shape should have.

But:

- How can we parameterize the convolution kernel for complex and arbitrary shapes (such as the handwritten numbers in MNIST)?

- How can we scale the parameterization when the number of classes to determine is very large and it is not feasible to determine it by hand?

Convolutional Neural Networks

To answer the two questions above, we must use the definition of a Convolutional Layer in the context of neural networks.

A Convolutional Layer is defined as the following mathematical operation:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

Where:

- Input is an input tensor (image).
- Out is the (known) output.
- Bias is a tensor of constants.
- Weight is a tensor of weights (the convolution tensor) that must be determined.
- C and N are the number of channels in the image and the number of images, respectively.

Convolutional Neural Networks

Therefore, we see that a convolutional layer is nothing more than the tensor formulation of the convolution integral.

In the previous topic, we saw that the tensor formulation could be expressed to define predefined shapes, but in this case of a Convolutional Neural Network, the elements to determine are the elements of the kernel tensor.

Since we have a tensor system of equations with unknown variables to determine, we can use supervised machine-learning techniques to determine them.

Since the convolution operation is in tensor format, and the natural formalism of neural networks is tensor operations, we can say that neural networks are the most natural and efficient formalism to determine the kernels of the basic shapes in images.

Convolutional Neural Networks

As an example, we will solve the same MNIST case that we solved with Support Vector Machine, but this time applying the Convolutional Neural Networks methodology.

To do so, we must determine the simplest topology we can propose.

The MNIST problem, from the point of view of the neural network formalism, is considered:

- A classification problem.
- Ten classes to determine: output dimension (10,)
- Input dimension (N, 28, 28).

Since this is a classification problem, considering a convolutional layer by itself is not enough to solve the problem, because the convolutional layer only finds patterns; it does not map those patterns to specific classes.

In addition, the output of the convolutional layer is a rank-2 tensor, while the output of a classifier has rank 1. We also need an 'adapter' that takes us from rank 2 to rank 1.

Convolutional Neural Networks

Convolutional layers have 4 configuration parameters that define them: kernel size, dilation, stride, and padding.

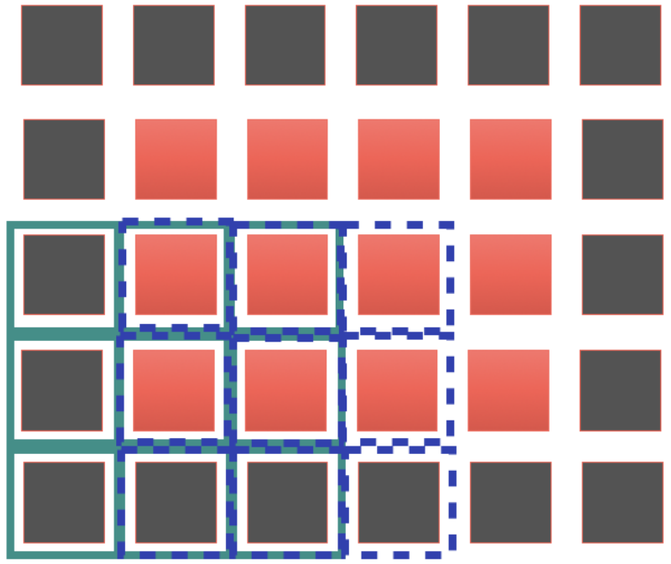


Illustration of a convolutional layer on a 4 x 3 pixel image with padding 1, stride 1, dilation 1, and a 3 x 3 kernel. Author's own work.

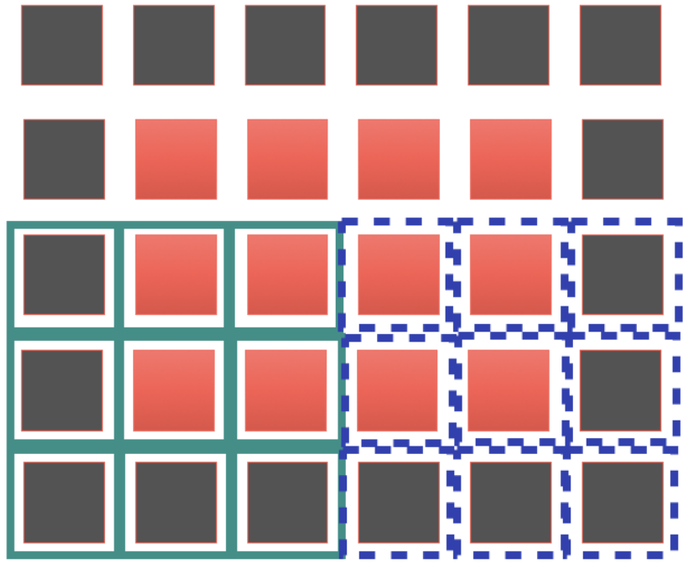


Illustration of a convolutional layer on a 4 x 3 pixel image with padding 1, stride 3, dilation 1, and a 3 x 3 kernel. Author's own work.

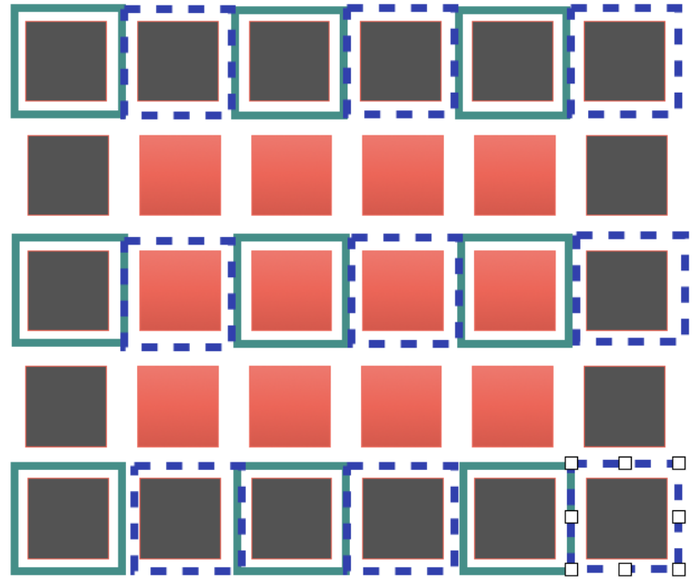
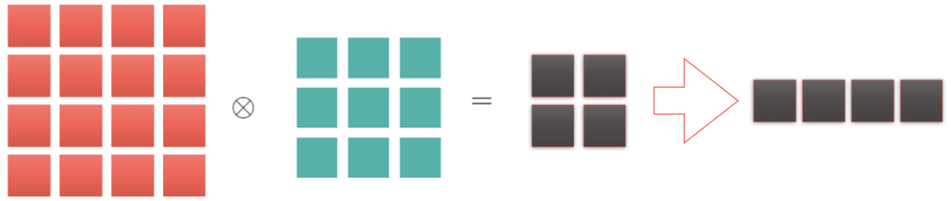


Illustration of a convolutional layer on a 4 x 3 pixel image with padding 1, stride 1, dilation 2, and a 3 x 3 kernel. Author's own work.

Convolutional Neural Networks

Convolutional layers reduce the spatial resolution of an image when applying the convolution operation.

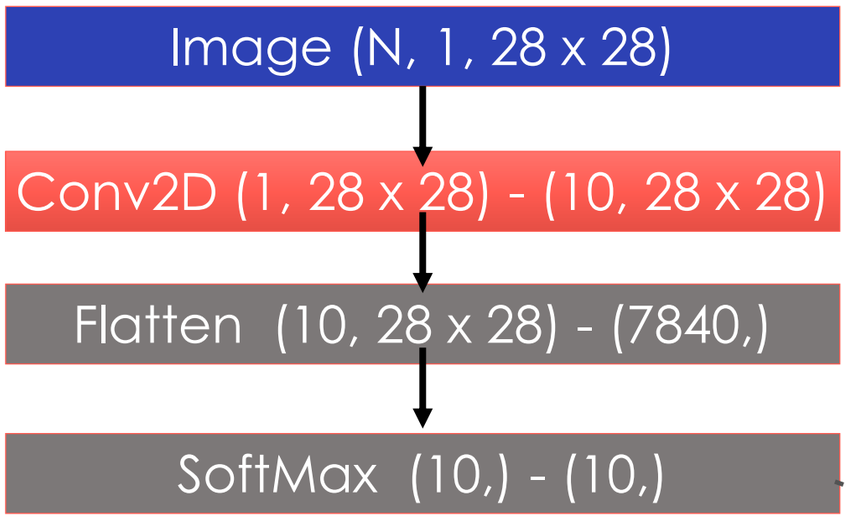
How much the spatial resolution (x) is reduced is defined by the hyperparameter configuration of each layer: kernel size (k), stride (s), and padding (p).

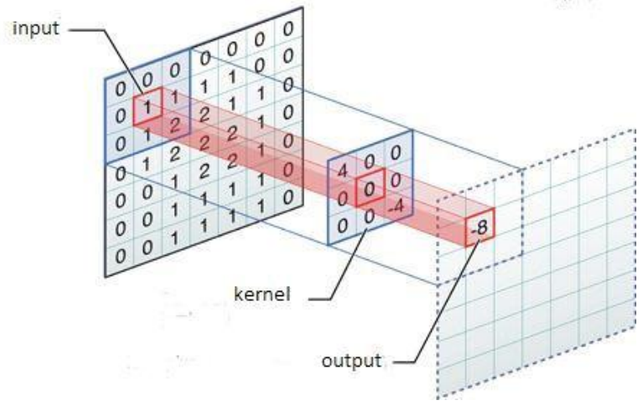


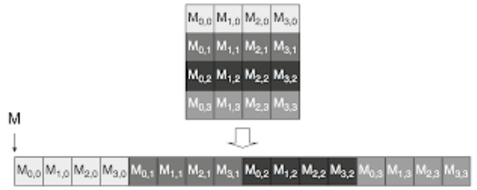
$$x_{out} = \text{int}\left(\frac{x_{in} + 2p - [(k-1) * d + 1]}{s}\right) + 1$$

Convolutional Neural Networks

The simplest topology that can be considered in neural networks for the MNIST problem is the following:



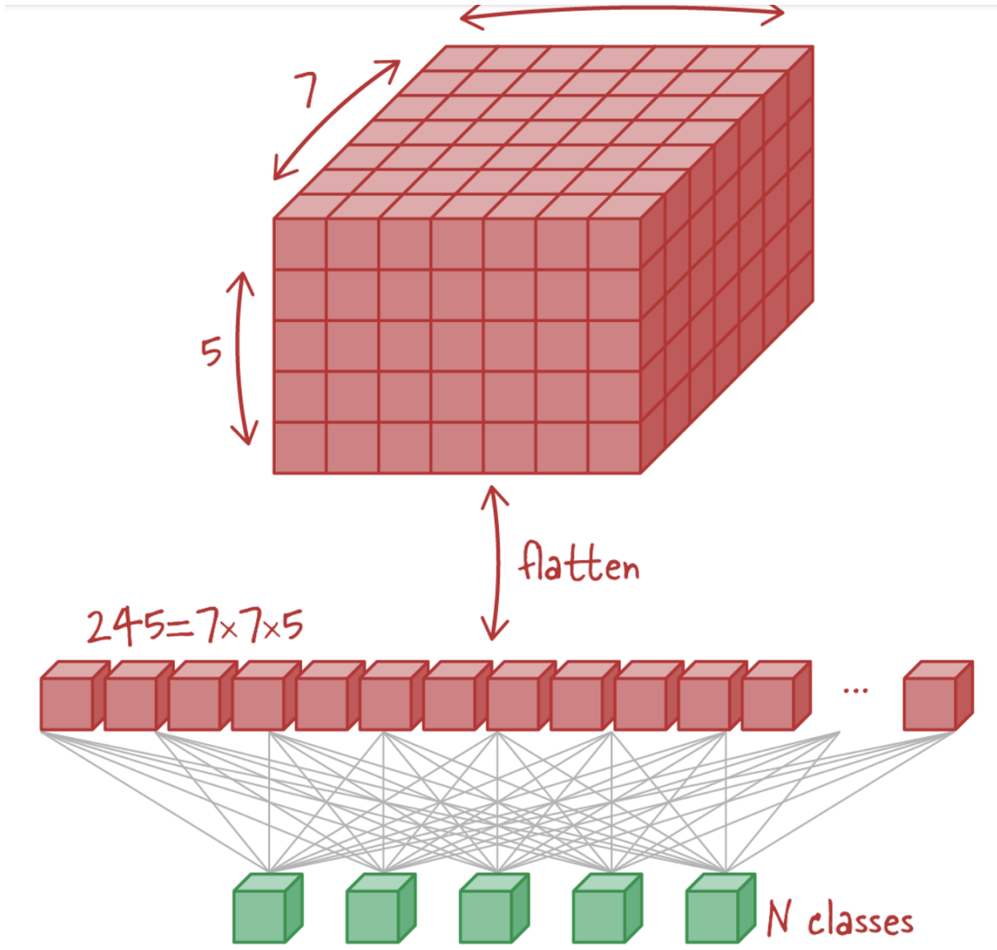




$$S(y_i) = \frac{e^{y_i}}{\sum e^{y_j}}$$

Convolutional Neural Networks

Graphical demonstration of the Flatten operation.



Convolutional Neural Networks

From the proposed topology, we must highlight two things:

- We only have 1 layer of weights (i.e., parameters) to be determined:
  A. The Conv2D layer (2D convolution), which fully performs the function of recognizing shapes and patterns in images.
- We have 2 layers whose only function is to perform mathematical transformations.
  B. Flatten, which takes us from the 2D space of the convolution to the 1D space of the classes.
  C. Softmax, which transforms the logits of the dense layer (i.e., numbers between -1 and 1) into the probability of belonging to a class by using exponentials.

Universidad Europea

Convolutional Neural Networks

In this case, if we look closely, we did not choose the number of output channels of the convolutional layer or the kernel size by chance. The kernel size is exactly the same size as the image, and the number of output channels is the same as the number of classes to determine.

The number of output channels is mathematically equivalent to the number of basic shapes the convolutional layer can determine. Each output channel constitutes a “filter” or “independent basic shape” that the neural network is able to recognize.

Thus, each output channel (or filter) can be interpreted as the average, or a representative prototype, of each of the classes to be determined.

In the case of MNIST, each output channel will be each of the basic digits from 0 to 9.

To make the example, we will implement it in PyTorch.

Universidad Europea

Introduction to PyTorch

Important note about execution environments.

Depending on the hardware it runs on, the Python version, the PyTorch version, and the operating system, we will have 3 different execution environments:

- CPU: on the regular PC CPU, mainly x86_64 architecture processors. Recently, support has been added for ARM CPUs for Apple's M1 and M2 Apple Silicon chips.
- CUDA: NVIDIA GPUs with support for CUDA execution cores.
- MPS: or Metal Performance Shaders. It is the functionality of the new Apple Silicon chips for running Artificial Intelligence.

The only requirement (given a compatible execution environment) is that all tensors are on the same device. That is, if my images are on the GPU, my labels must also be on the GPU and the Neural Network weights must also be on the GPU.

Convolutional Neural Networks

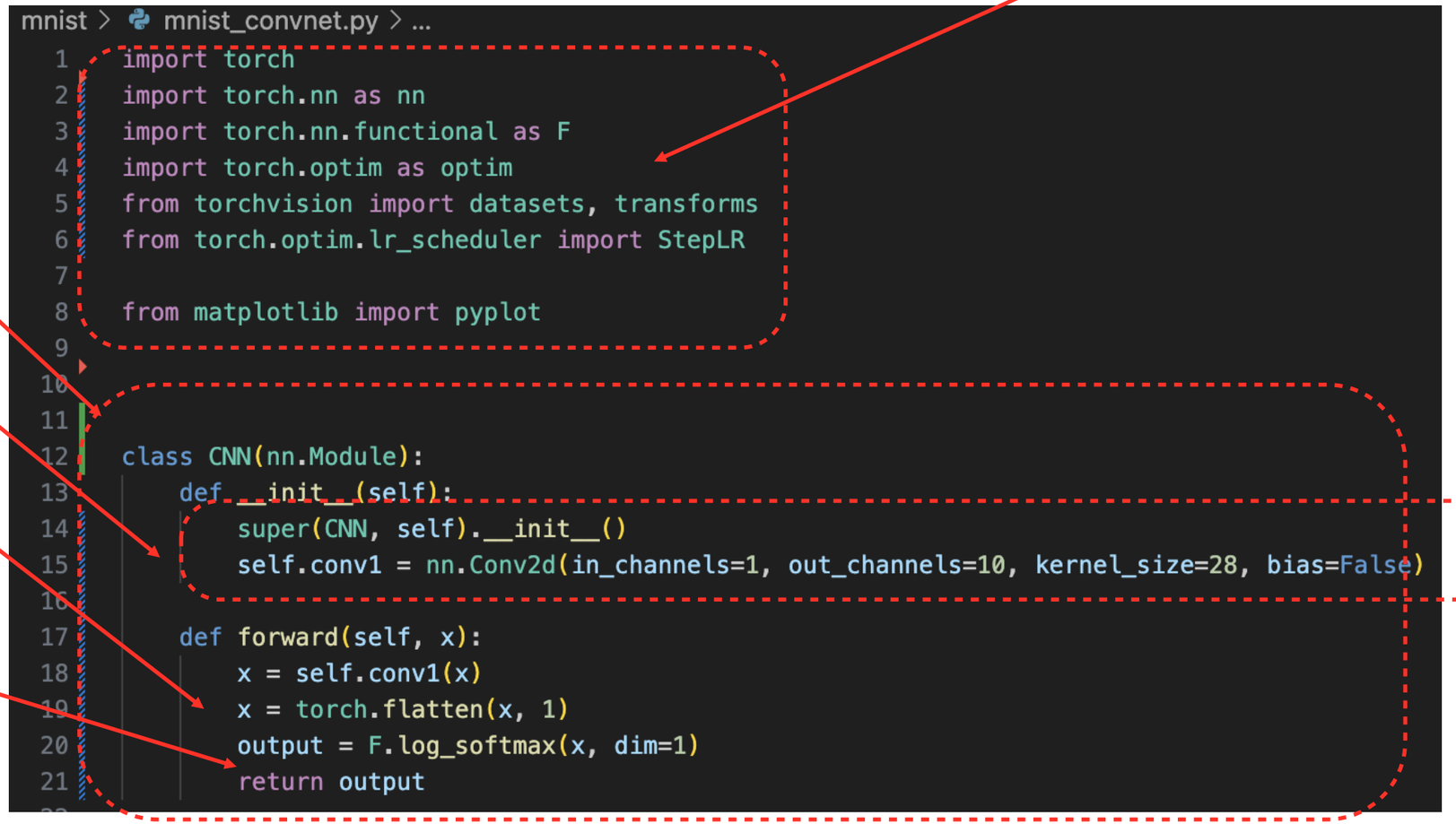
We import libraries

We define the Neural Network topology

We apply a Conv2D layer

We apply a flatten to go from a rank-2 tensor to rank 1

We apply a softmax as the final activation layer to turn it into a classifier



Convolutional Neural Networks

We put the model in Train mode to update the gradients of the neural network weight tensors

We perform an inference

We update the gradients of the neural network tensors

```python
def train(model, device, train_loader, optimizer, epoch):
    model.train()
    running_loss = 0.
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % 1000 == 0:
            print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))

        running_loss += loss.item()

    return running_loss
```

We iterate over the batches

We send the tensors to the GPU/CPU

We zero the gradients

We compute the loss

We advance the optimizer one step

Convolutional Neural Networks

We define the device. In this case, Apple Silicon MPS

We load the MNIST data into a Dataset

We instantiate the model

We perform N iteration cycles per epoch

```python
if __name__ == '__main__':

    device = torch.device("mps")

    transform=transforms.Compose([
        transforms.ToTensor()
        ])
    dataset1 = datasets.MNIST('./MNIST', train=True, download=True,
                        transform=transform)
    dataset2 = datasets.MNIST('./MNIST', train=False,
                        transform=transform)
    train_loader = torch.utils.data.DataLoader(dataset1, batch_size=64)
    test_loader = torch.utils.data.DataLoader(dataset2, batch_size=64)

    model = CNN().to(device)
    optimizer = optim.Adadelta(model.parameters(), lr=1.0)

    loss_list = list()
    accuracy_list = list()

    scheduler = StepLR(optimizer, step_size=1, gamma=0.7)
    for epoch in range(1, 15 + 1):
        loss = train(model, device, train_loader, optimizer, epoch)
        loss_list.append(loss)
        accuracy = test(model, device, test_loader)
        accuracy_list.append(accuracy)
        scheduler.step()
```

We define which transformations to apply to the images. In this case, only convert to Tensor

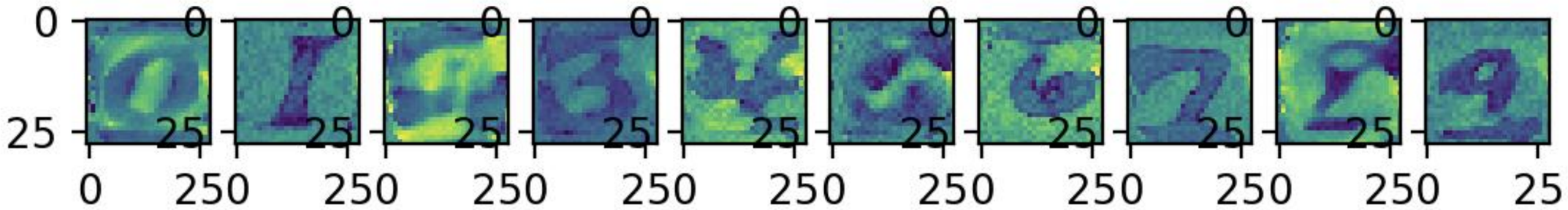
We define the DataLoader to iterate in batches

We define the optimizer to use in the training process

We define a decaying learning rate to improve the convergence of the algorithm.

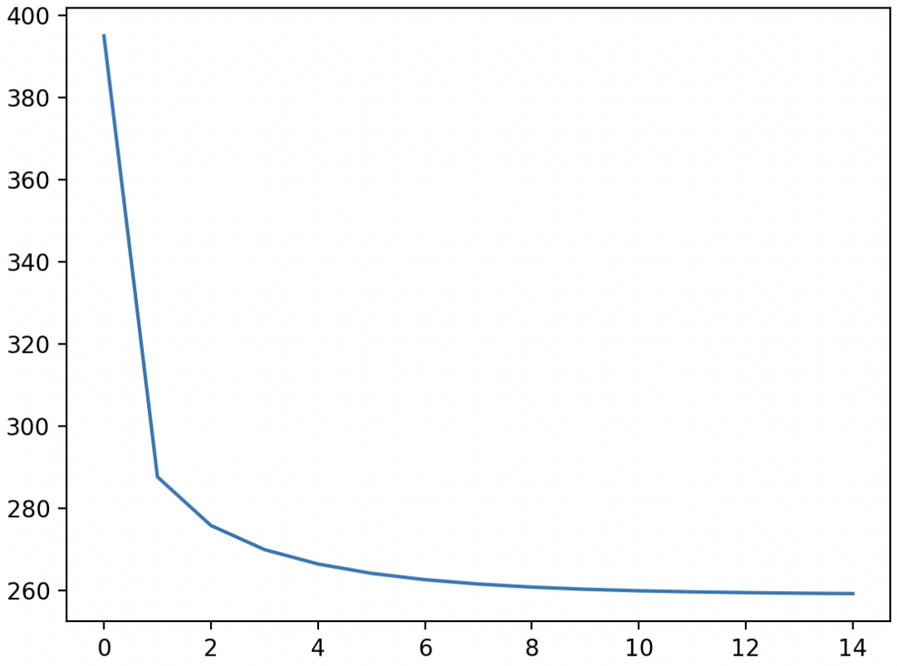
Convolutional Neural Networks

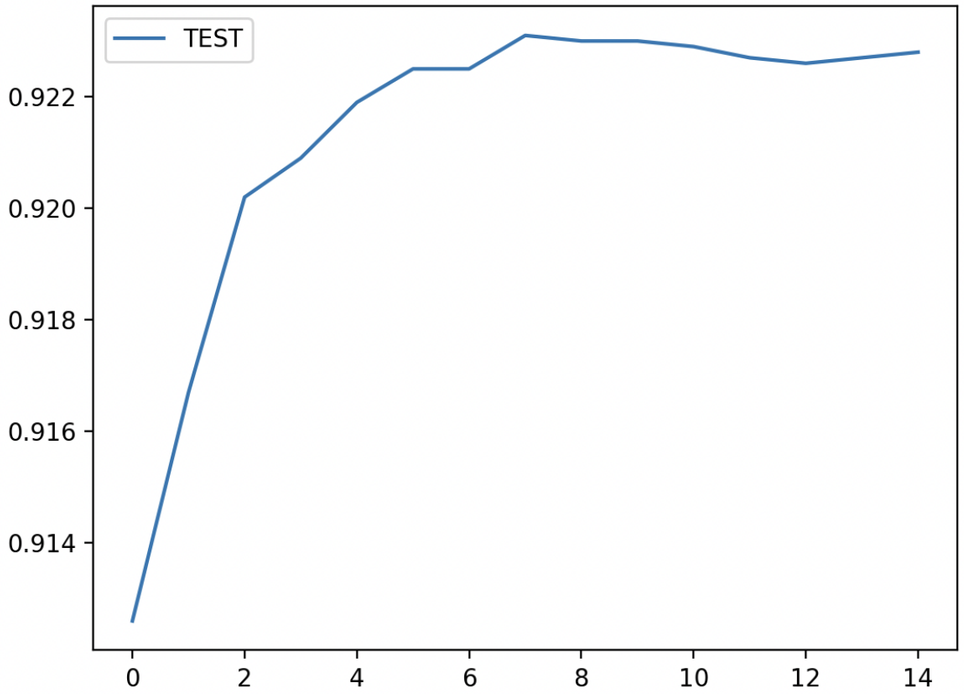
We can see that each of the channels of the convolutional layer of the Neural Network looks like an average of the images.



Convolutional Neural Networks

Here we see the accuracy and the loss.





02

Deep Learning

Deep Learning

To understand the fundamentals of Deep Learning applied to Computer Vision, it is necessary to understand the Generalized Fourier Theorem.

Let there be a set of functions over the real numbers:

$$\Phi = \{\phi_n : \mathbb{R} \to \mathbb{R}\}_{n=0}^{\infty}$$

Such that these functions are orthogonal to each other:

$$\langle \phi_i, \phi_j \rangle = \int_{-\infty}^{\infty} \phi_i(x)\phi_j(x)dx; \forall i \neq j$$

These functions will form a vector space, so that the set of orthogonal functions φ constitutes a basis of the vector space.

Deep Learning

Then, any function over the real numbers can be expressed as a linear combination (with constant coefficients c) of that set of orthogonal functions.

$$f(x) = \sum_{n=0}^{\infty} c_n \phi_n(x); c_n = \frac{\langle f(x), \phi_n(x))}{\langle \phi_n(x), \phi_n(x) \rangle}$$

That is, any function f can be expressed as a linear combination of the elements that constitute the basis of the vector space.

Deep Learning

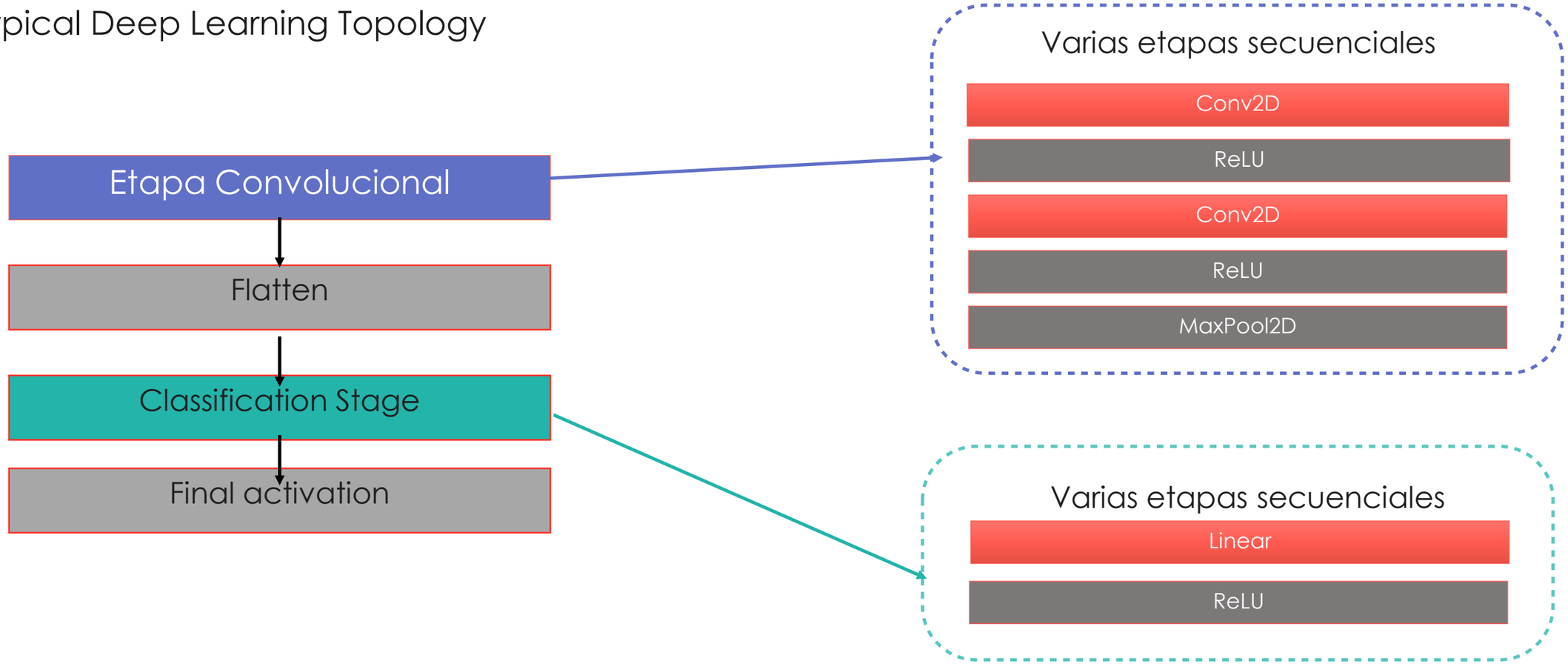
That said, the function f will be the image, the coefficients c will be the results of applying the convolutions of the convolutional layers to the image, and the functions φ will be the kernels determined by the convolutional layers.

Therefore, we can say that applying Deep Learning to a Computer Vision problem is mathematically equivalent to performing a Fourier decomposition.

In the case of a classification problem, the different coefficients of the Fourier decomposition pass through a "classification head", which, through a combination of Linear layers, assigns a class to a set of coefficient values from the Fourier decomposition.

Deep Learning

Typical Deep Learning Topology



Deep Learning

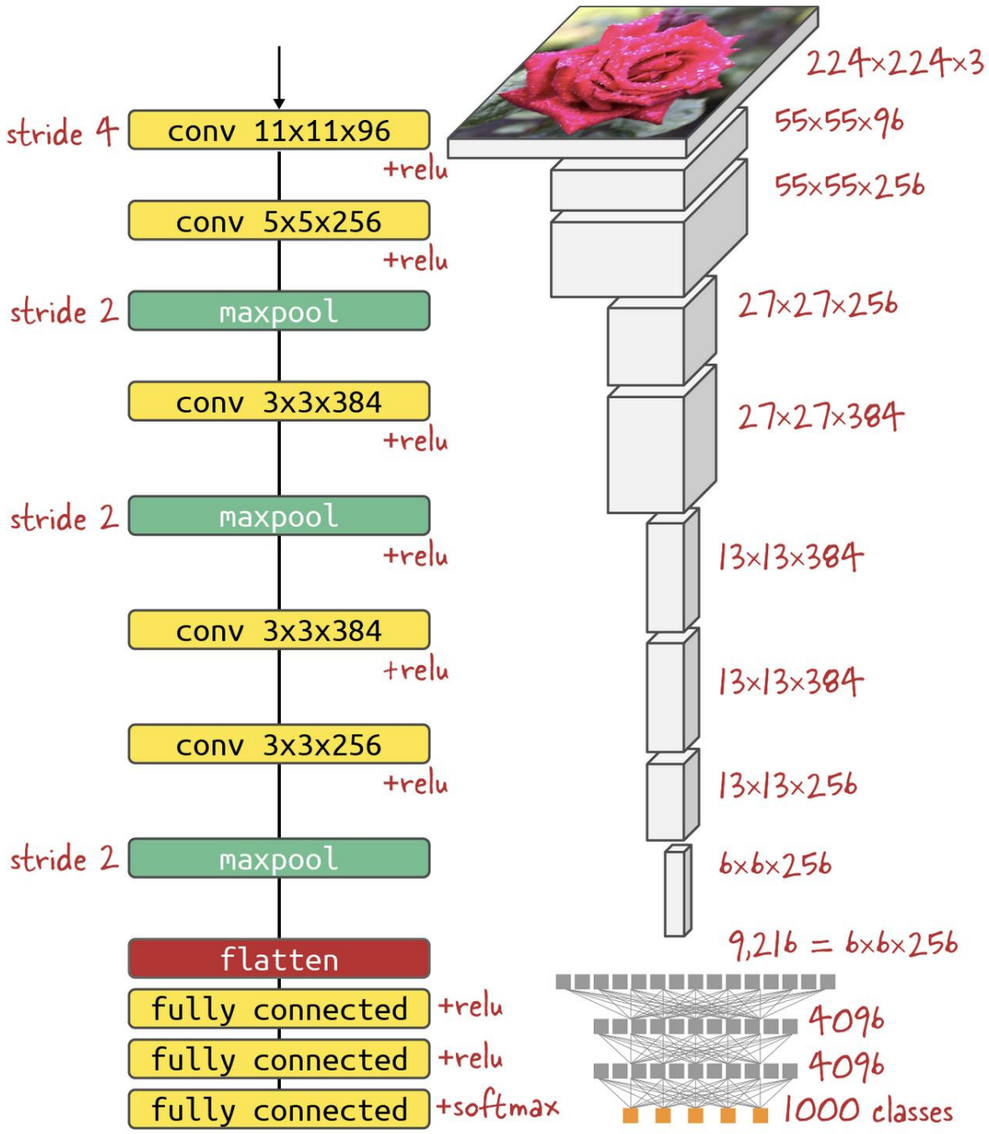
Graphical representation of a Deep Learning topology.

In this case, AlexNet.

Visually, the concatenation of convolutional layers with small kernels and many output channels can be appreciated.

AlexNet (year 2012) is the first deep neural network applied to Computer Vision and has 3.7M parameters.

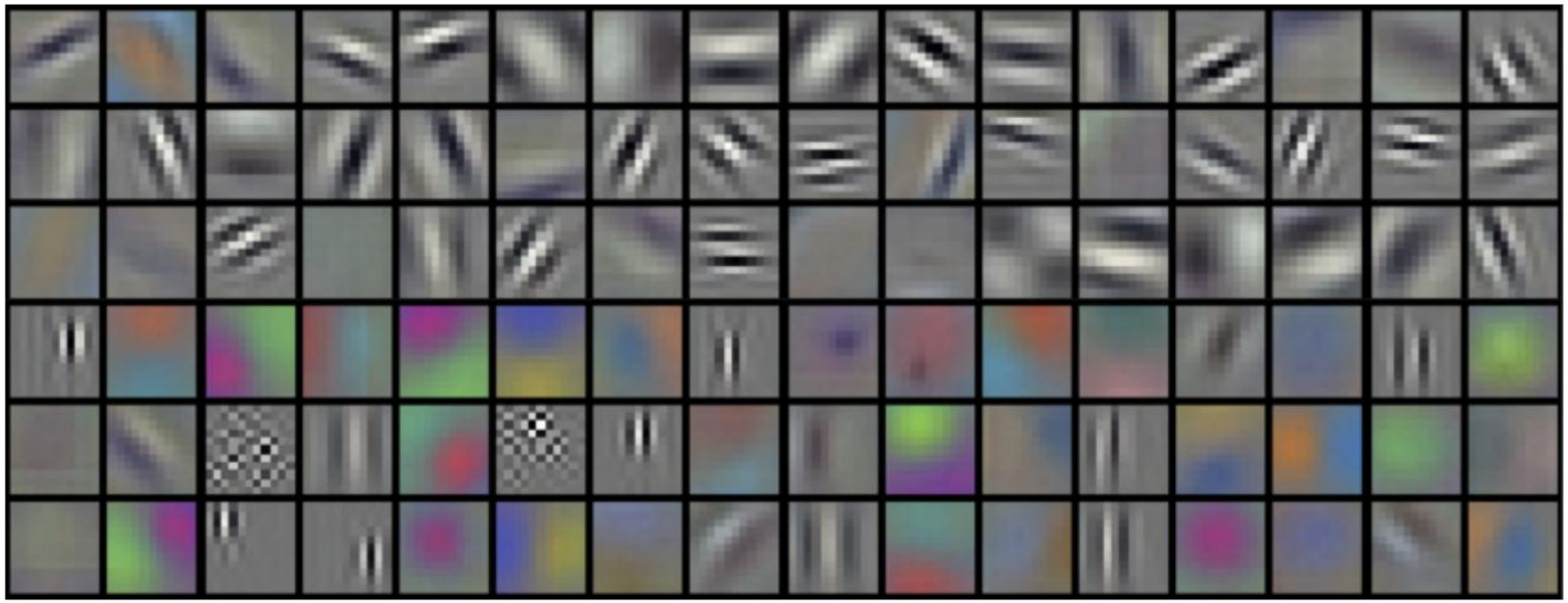
https://papers.nips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf



Deep Learning

Complete set of filters from the first layer of AlexNet. You can see that they are elementary shapes, such as vertical lines, horizontal lines, checkerboard patterns…

Returning to the parallel with the convolution integral, these will be the basic shapes that will form the basis of our vector space of elementary shapes. Shapes from which we will describe any image.



Deep Learning

As with any Deep Learning model, certain mathematical tricks will be needed to facilitate the convergence of the algorithm and avoid overfitting.

To avoid overfitting, various techniques known as regularization are needed, including:

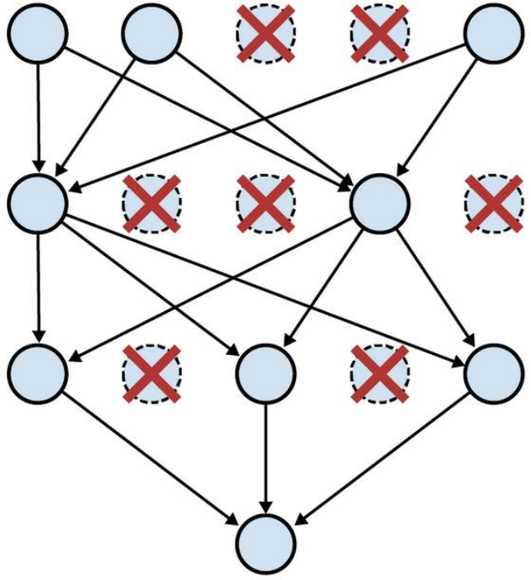
- Dropout.
- Batch Normalization.
- Pooling Layers (typically MaxPool or AvgPool).

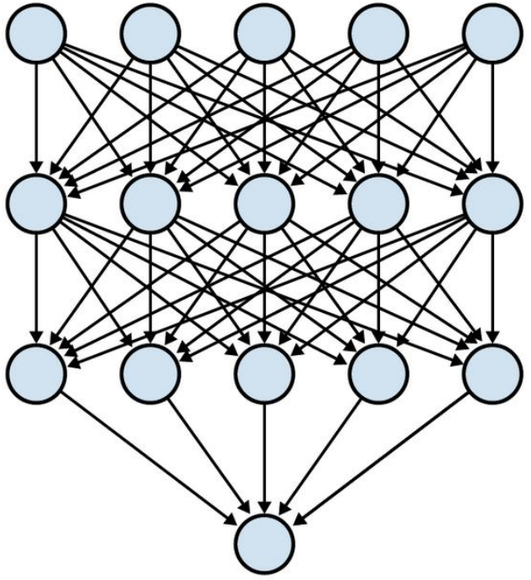
Deep Learning

DROPOUT

Dropout is a technique that consists of setting to zero some elements of the tensors of a neural network layer at random (typically 20%–50%).

The goal is to prevent specific groups of tensor elements from memorizing the data.



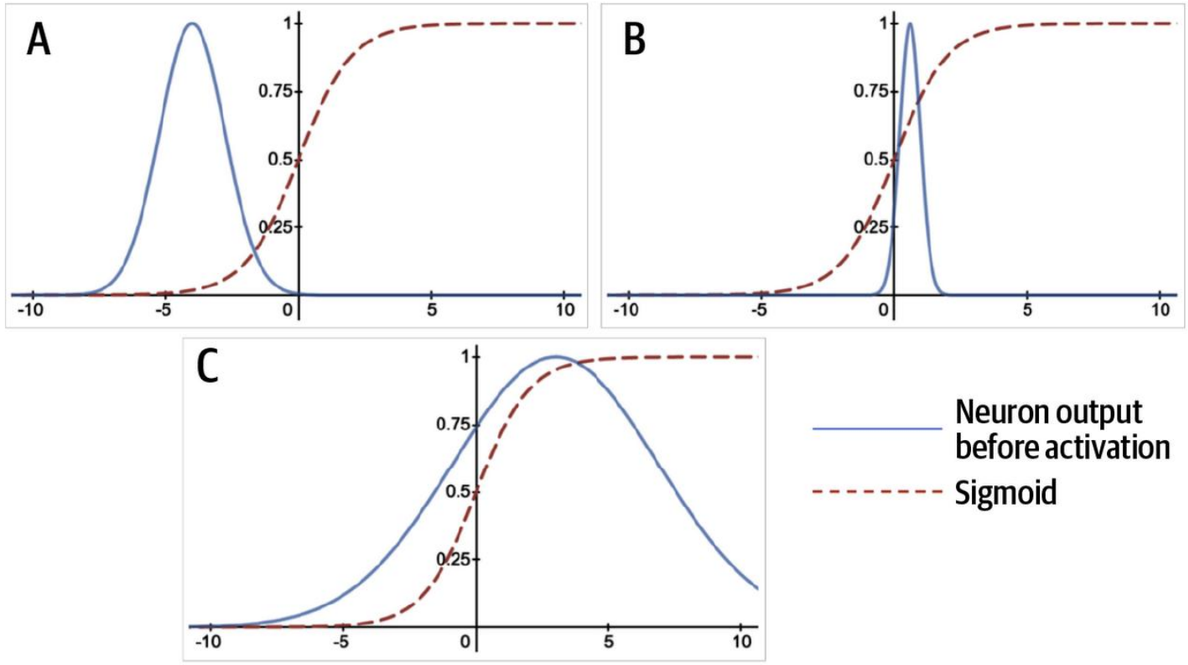


Deep Learning

BATCH NORMALIZATION

Many times, after passing through many layers of a neural network, the output values can fall outside the range of values from which the activation layer sets the parameters to zero (A).

Batch Normalization seeks to bring the values into a range in which the activation function has an effect both on the position of the values (B) and on their amplitude/distribution (C).

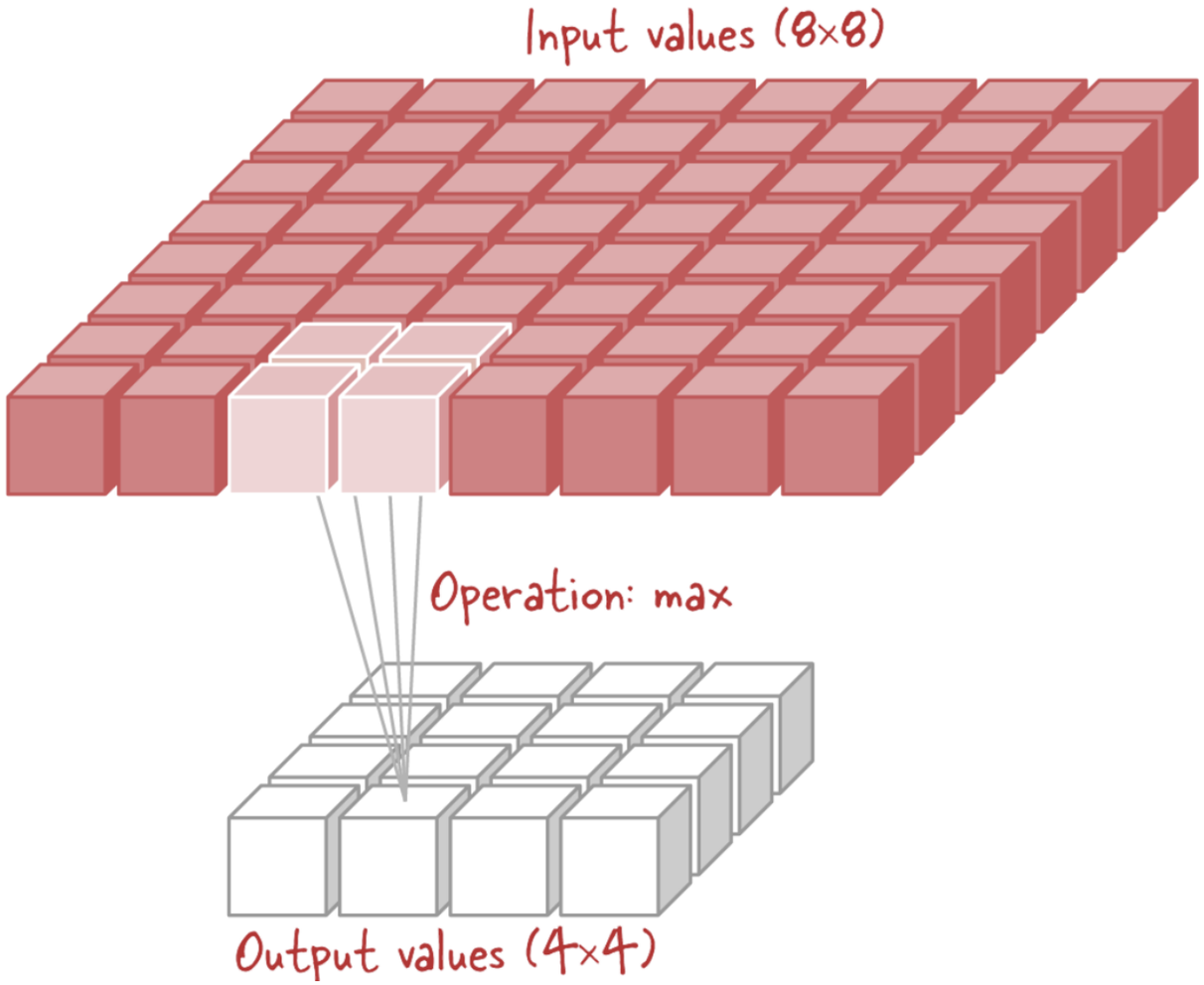


Deep Learning

POOLING LAYERS

Applying a Pooling Layer is a downsampling technique to reduce the dimensionality of the problem.

To do so, the pixels of one of the intermediate layers after a convolutional layer are grouped and an aggregation operation is applied, which can be the maximum or the mean, namely MaxPool and AvgPool respectively.

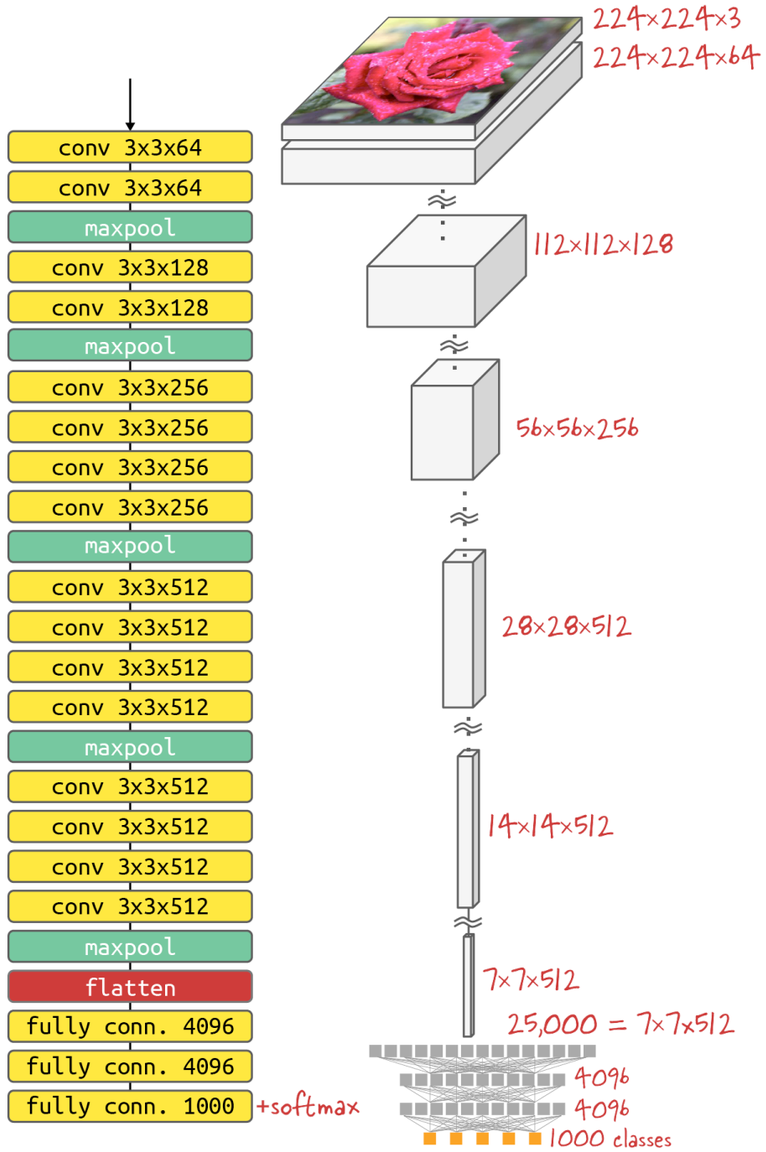


Deep Learning

Other topologies include VGG-19, shown on the side, which has a topology similar to AlexNet but has more layers and more parameters (20M).

It is a model from the year 2014

https://arxiv.org/abs/1409.1556



Deep Learning

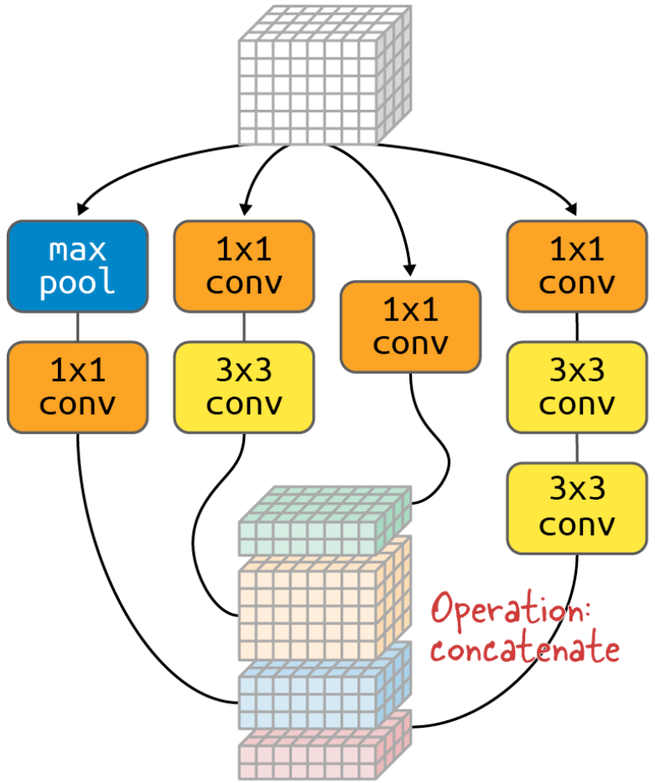
The InceptionV3 topology has a non-sequential architecture that combines nested elements from different layers.

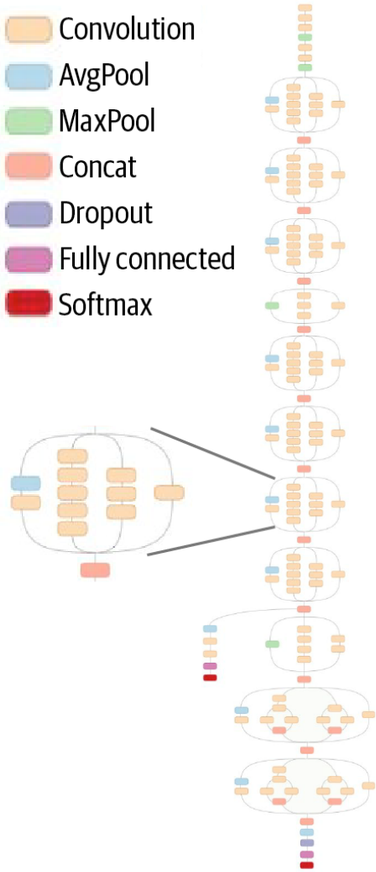
It departs from the norm of topologies that simply chain convolutional layers in a linear fashion.

It has 22M parameters.

It is a model from the year 2015.

https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Szegedy_Rethinking_the_Inception_CVPR_2016_paper.pdf





Deep Learning

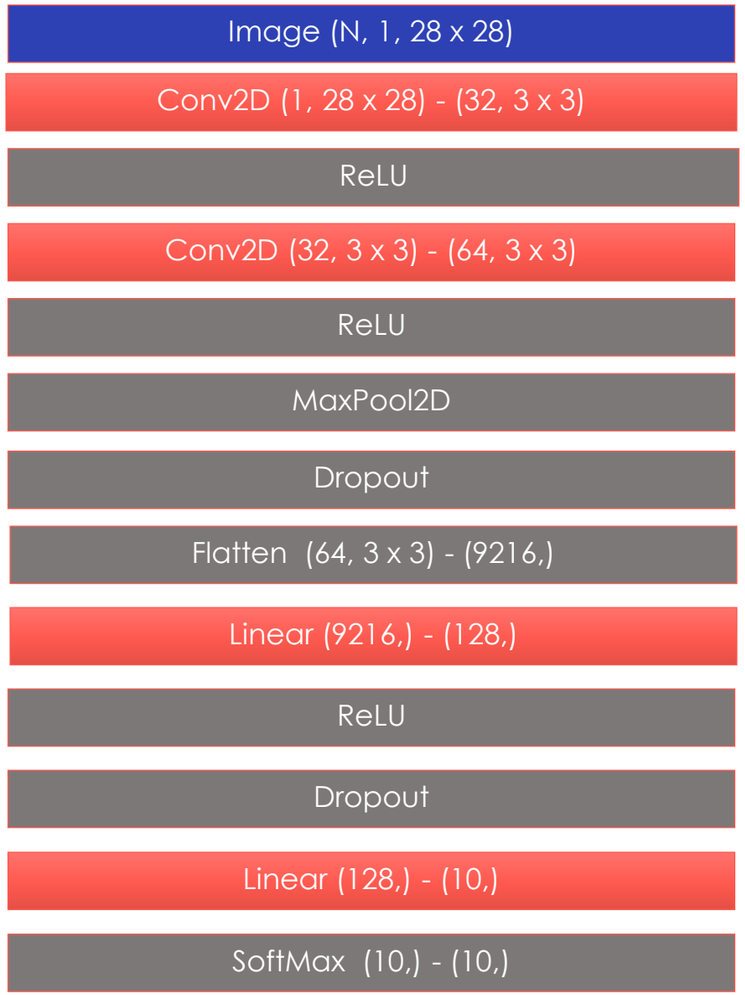
Applying Deep Learning to a Computer Vision problem ultimately is a trade-off between having as many parameters as possible and, at the same time, having a finite training dataset—while still generalizing enough not to be limited only to things bounded within the training dataset.

There are numerous topologies with different approaches. There is no rule of thumb for which one to choose. You have to try.

Coming up with a topology that is good and goes beyond what has already been invented is a combination of intuition, art, and luck.

Model	Parameters (excl. classification head[a])	ImageNet accuracy	104 flowers F1 score[b] (fine-tuning)
EfficientNetB6	40M	84%	95.5%
EfficientNetB7	64M	84%	95.5%
DenseNet201	18M	77%	95.4%
Xception	21M	79%	94.6%
InceptionV3	22M	78%	94.6%
ResNet50	23M	75%	94.1%
MobileNetV2	2.3M	71%	92%
NASNetLarge	85M	82%	89%
VGG19	20M	71%	88%
Ensemble	79M (DenseNet210 + Xception + EfficientNetB6)	-	96.2%

Deep Learning



For the proposed Deep Learning topology with MNIST, we will fit a Neural Network with more than one hidden layer.

To do so, we will consider the simplest topology:

- Two convolutional layers, with a small kernel but many channels.
- Two Linear layers (one of them hidden) to act as a classifier.

With many layers, we run into issues that commonly arise in Deep Learning, so we need to include:

- ReLU activation to capture nonlinearities.
- Dropout to avoid overfitting.
- MaxPool to avoid overfitting.

Deep Learning

We declare 2 convolutional layers with multiple channels and a small kernel

We declare a final linear layer with 10 output neurons (one per class)

We apply a pooling layer every two conv layers

```
10
11  class Net(nn.Module):
12      def __init__(self):
13          super(Net, self).__init__()
14          self.conv1 = nn.Conv2d(1, 32, 3, 1)
15          self.conv2 = nn.Conv2d(32, 64, 3, 1)
16          self.dropout1 = nn.Dropout(0.25)
17          self.dropout2 = nn.Dropout(0.5)
18          self.fc1 = nn.Linear(9216, 128)
19          self.fc2 = nn.Linear(128, 10)
20
21      def forward(self, x):
22          x = self.conv1(x)
23          x = F.relu(x)
24          x = self.conv2(x)
25          x = F.relu(x)
26          x = F.max_pool2d(x, 2)
27          x = self.dropout1(x)
28          x = torch.flatten(x, 1)
29          x = self.fc1(x)
30          x = F.relu(x)
31          x = self.dropout2(x)
32          x = self.fc2(x)
33          output = F.log_softmax(x, dim=1)
34          return output
```

We declare two Dropout layers

We declare a hidden (hidden-layer) Linear layer

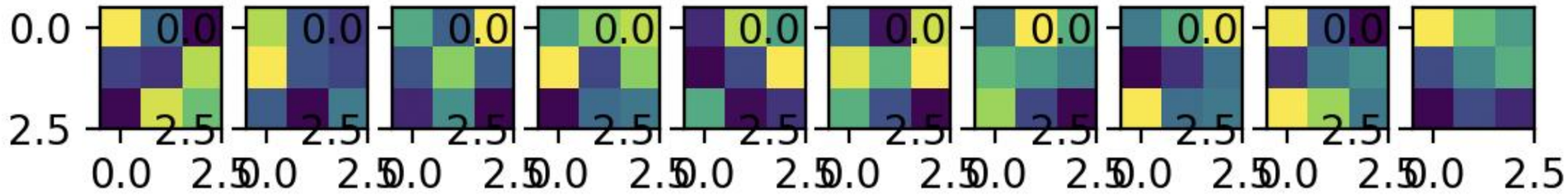
After each layer we add a ReLU activation

Dropout after the convolutional part and after the Hidden layer.

Deep Learning

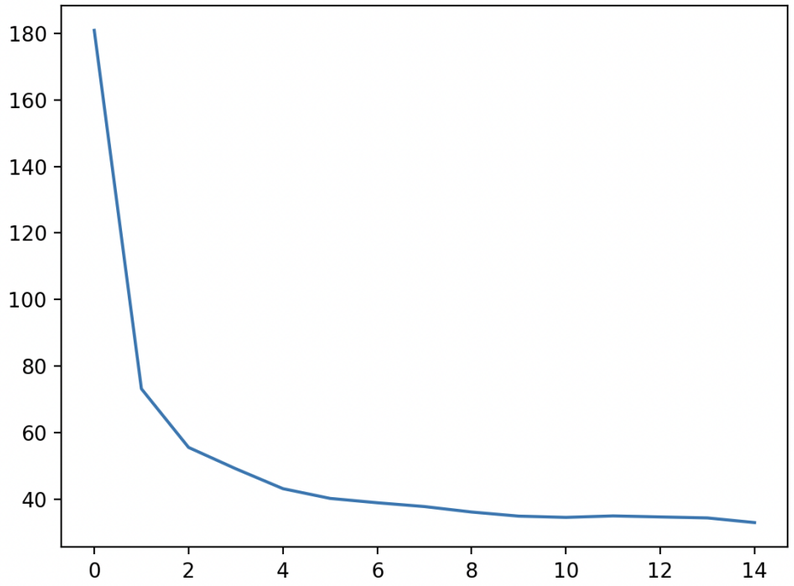
We can see some of the channels of the first convolutional layer of the Neural Network.

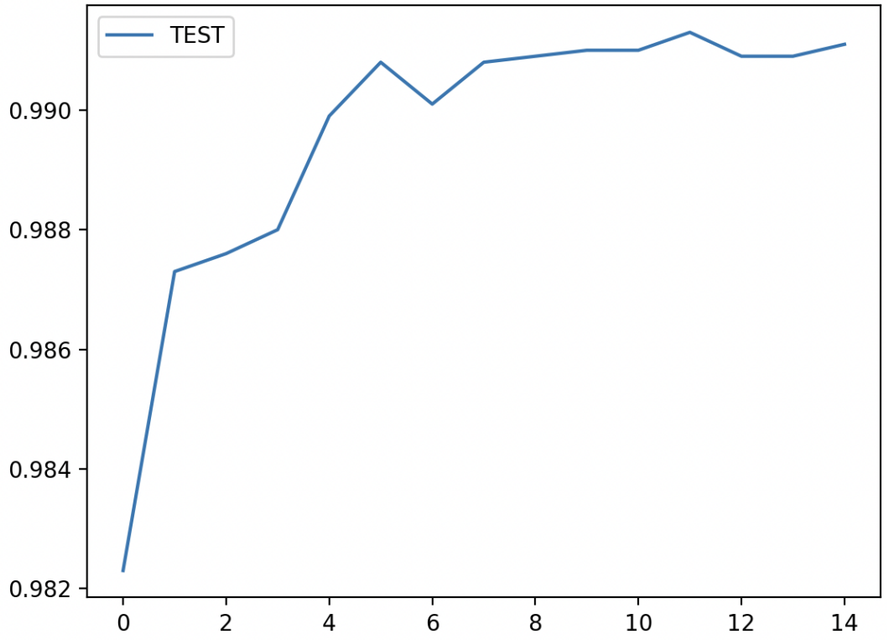
We can see that they are smaller kernels (3 x 3 pixels) and that they lose any interpretability relationship with the original image.



Deep Learning

Here we see the accuracy and the loss.





Deep Learning

A notable aspect of switching from using a single convolutional layer in the MNIST case is that it yields better convergence in terms of the loss function and also achieves higher accuracy.

This is due to two clearly differentiated factors. On the one hand, by using many channels but a smaller kernel size, the Neural Network is able to generalize better.

On the other hand, by using Deep Learning methods such as Dropout or pooling layers and ReLU activation layers, convergence improves overall and more complex patterns can be detected.

03

Transfer Learning

Transfer Learning

Throughout the topic we have seen that the most general way to tackle a Computer Vision problem (especially a classification problem) is to use Deep Neural Networks (Deep Learning) in which the more convolutional layers, the better.

Likewise, we have also seen that the more convolutional layers, the many more parameters to estimate, and therefore many more images will be needed.

Training a Computer Vision Deep Learning algorithm is expensive in terms of compute time, but also expensive in terms of dataset construction regarding size and image labeling.

However, we know that Deep Learning Computer Vision topologies have two clearly differentiated parts.

- **The Convolutional part, which is responsible for recognizing generic shapes and patterns.**
- **The Classifier part, which is responsible for, based on the generic shapes, classifying into specific classes.**

Transfer Learning

In the pretrained algorithms we use, whether used as-is or to apply Transfer Learning techniques, we must keep in mind that they have a predefined topology and dimensions.

This implies that the input image tensors are expected to have fixed dimensions, so we must resize our images accordingly.

- If the original image has a higher resolution (in terms of pixels), we will lose quality and information. We must evaluate whether that is important or not. Nowadays we will almost always find ourselves in the higher-resolution situation.

- If the original image has a different number of channels than expected, it must be handled carefully. If the pretrained network expects 3 channels but our images are black and white (1 channel), we must replicate the channel three times.

Transfer Learning

Transfer Learning applied to Computer Vision consists of:

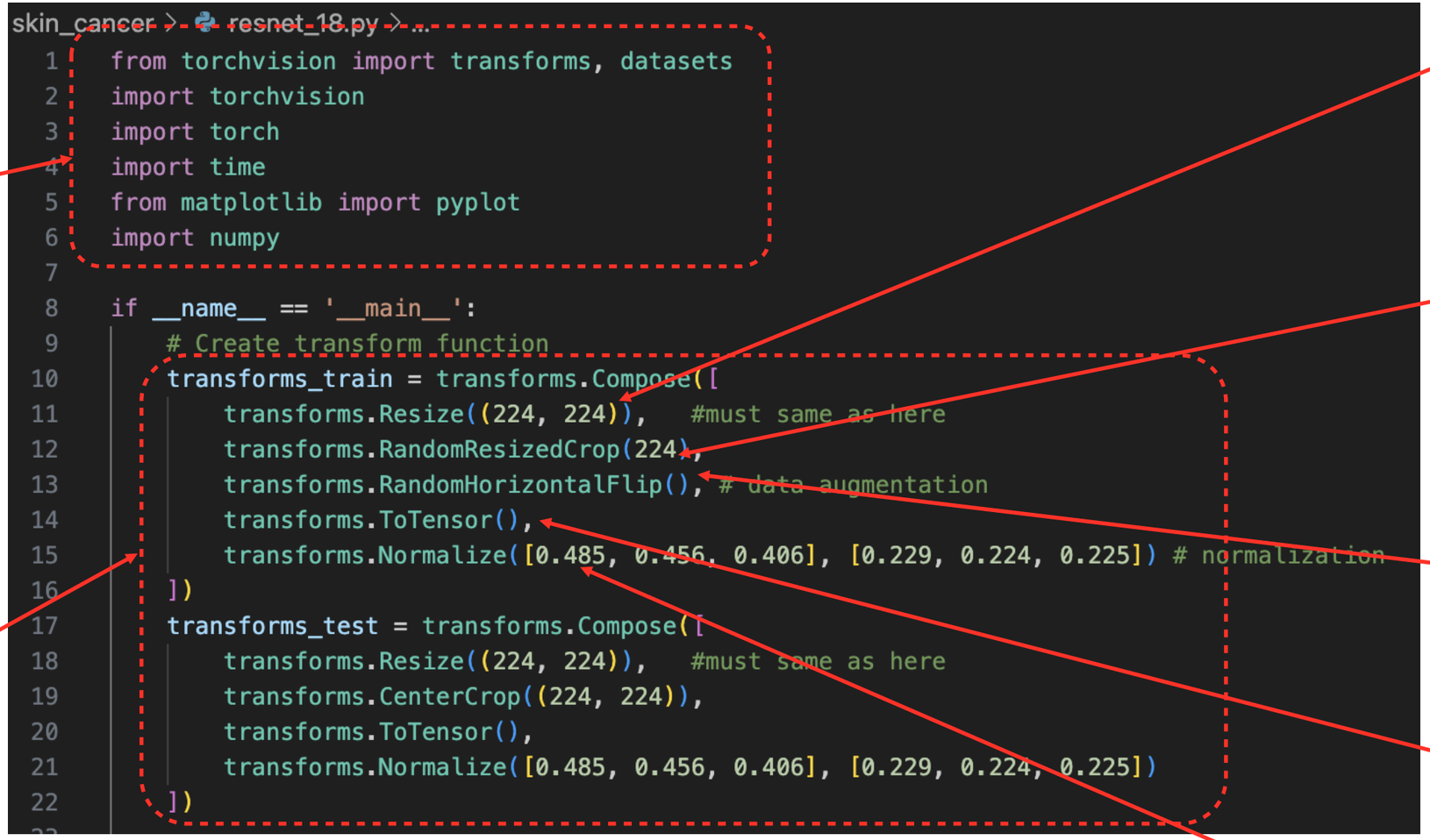
- Taking advantage of a Deep Learning topology (e.g., ResNet) that is already pretrained on a very large and generic dataset.
- Freezing the weights of the Convolutional part.
- Retraining the classifier layers (generally Linear layers) to adapt them to the classes to be determined in our problem and to our dataset.

Let's see it applied to the Skin Cancer dataset

Transfer Learning

We import the libraries

We define the transformations of the training and test datasets



Includes a Resize to the size expected by ResNet18

Creates a random crop on some images

Creates a random mirror reflection on some images

Transforms to Tensor

Normalizes the tensor

Transfer Learning

We read the image folders

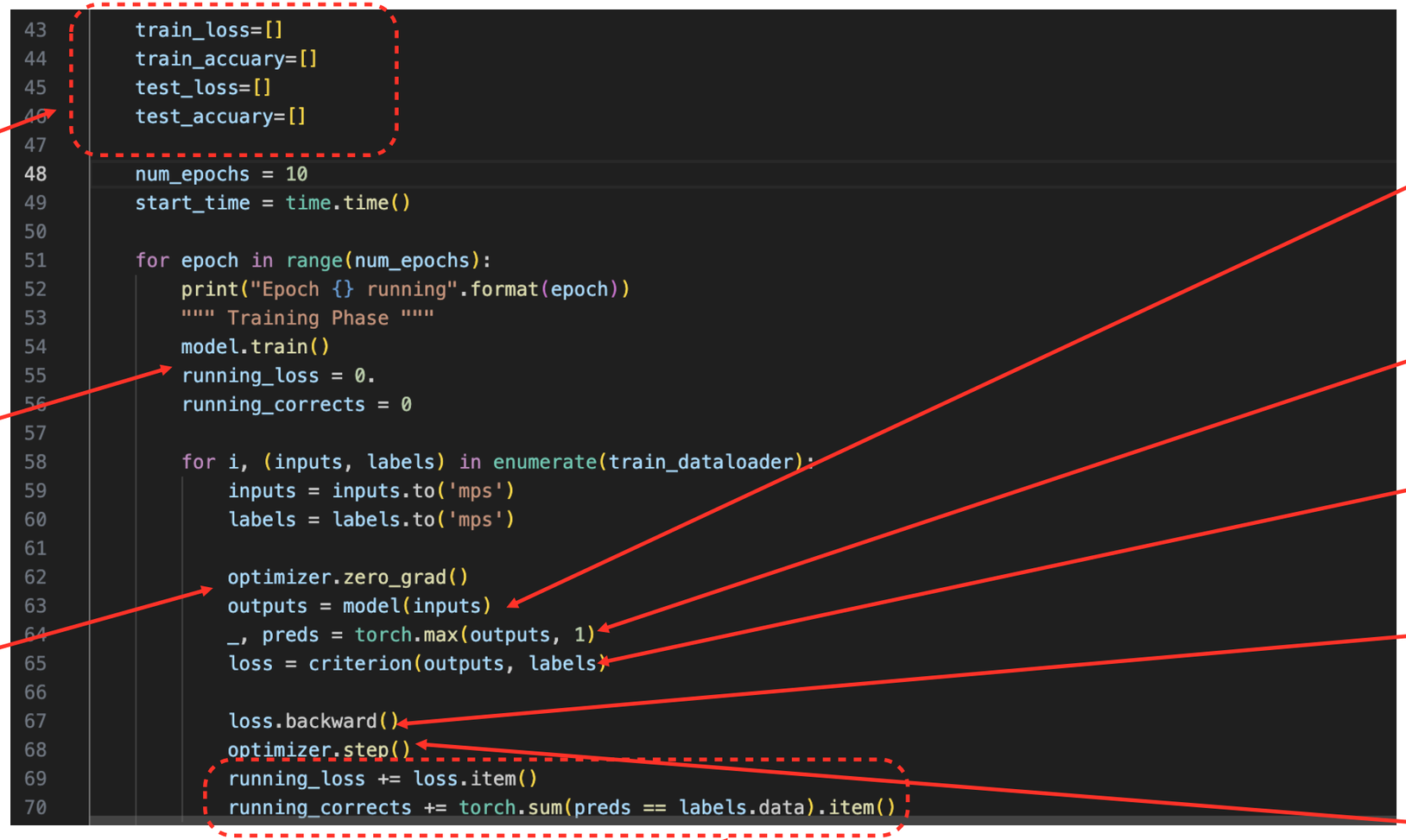
We transform into a DataLoader so we can read in batches

```
25  train_dir = "./skin_cancer/data/train/"
26  test_dir = "./skin_cancer/data/test/"
27  train_dataset = datasets.ImageFolder(train_dir, transforms_train)
28  test_dataset = datasets.ImageFolder(test_dir, transforms_test)
29
30  train_dataloader = torch.utils.data.DataLoader(train_dataset, batch_size=12, shuffle=True, num_workers=0)
31  test_dataloader = torch.utils.data.DataLoader(test_dataset, batch_size=12, shuffle=False, num_workers=0)
32
33  model = torchvision.models.resnet18(pretrained=True)
34  num_features = model.fc.in_features
35  print(num_features)
36
37  model.fc = torch.nn.Linear(512, 2)
38  model = model.to('mps')
39  criterion = torch.nn.CrossEntropyLoss()
40  optimizer = torch.optim.SGD(model.parameters(), lr=0.0001, momentum=0.9)
41
```

We instantiate the ResNet18 model and download its pretrained weights; we identify the size of the last layer

We replace the last layer with a new Linear layer with 2 classes (Benign/Malignant). We define the loss function and associate the optimizer with the model

Transfer Learning



We define where to store the metrics

We set the model to train mode

We zero the gradient

We perform inference

We obtain the results

We compute the loss

We backpropagate the gradients

We step the optimizer

We obtain metrics

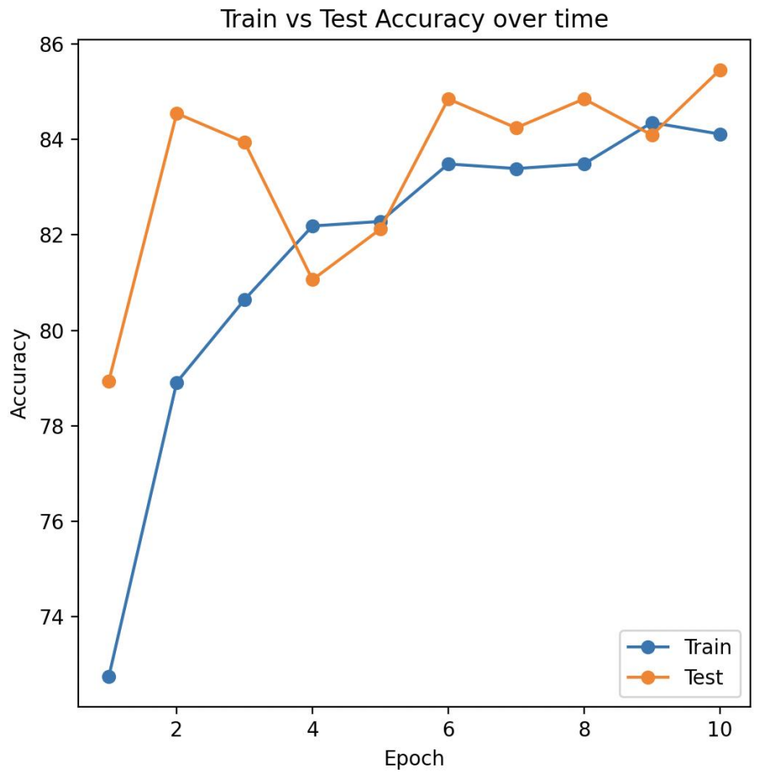
Transfer Learning

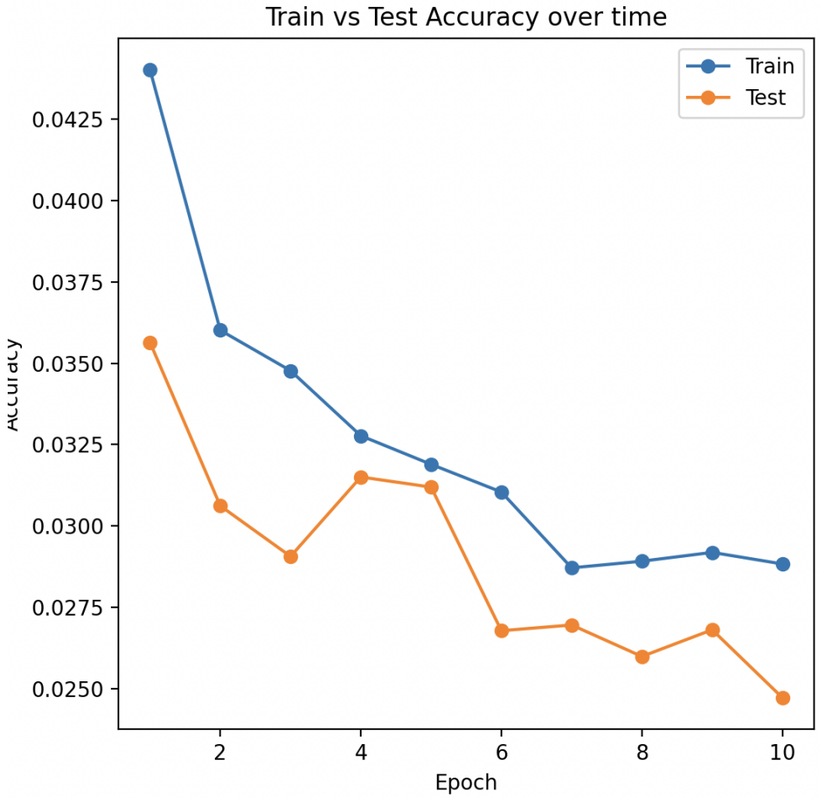
We obtain the metrics on the test dataset

```
71
72      epoch_loss = running_loss / len(train_dataset)
73      epoch_acc = running_corrects / len(train_dataset) * 100.
74
75      train_loss.append(epoch_loss)
76      train_accuary.append(epoch_acc)
77      # Print progress
78      print('[Train #{}] Loss: {:.4f} Acc: {:.4f}% Time: {:.4f}s'.format(ep
79
80      model.eval()
81      with torch.no_grad():
82          running_loss = 0.
83          running_corrects = 0
84          for inputs, labels in test_dataloader:
85              inputs = i   (variable) model: ResNet
86              labels = l
87              outputs = model(inputs)
88              _, preds = torch.max(outputs, 1)
89              loss = criterion(outputs, labels)
90              running_loss += loss.item()
91              running_corrects += torch.sum(preds == labels.data).item()
92          epoch_loss = running_loss / len(test_dataset)
93          epoch_acc = running_corrects / len(test_dataset) * 100.
94
95          test_loss.append(epoch_loss)
96          test_accuary.append(epoch_acc)
97
```

Transfer Learning

Here we can see the accuracy and the loss.





04

Image Segmentation

Object identification in images

In the first topic we saw, using the convolution integral, how to detect basic shapes within images, while in the previous topic we studied how to classify images as a whole by applying Deep Learning.

In this topic we will see how to identify objects within images using Deep Learning.

Within object identification tasks we find 3 clearly differentiated types:

- **Semantic Segmentation: consists of identifying types of objects without individually distinguishing objects belonging to the same class. That is, if we identify people, the algorithm detects all people as belonging to that class, but it is not able to detect different people from one another. These types of algorithms are based on identifying image textures and are especially useful for classifying and detecting uncountable and continuous objects such as roads, forest masses, water…**
- **Instance Segmentation: consists of detecting objects within images while distinguishing different individuals as distinct realizations of the same class. The disadvantage of these algorithms is that they are not able to detect uncountable objects, such as water or roads.**
- **Panoptic Segmentation: is capable of simultaneously performing Instance Segmentation and Semantic Segmentation tasks on the same image, always giving priority to Instance Segmentation. It is the most recent algorithm and dates from 2019 (https://openaccess.thecvf.com/content_CVPR_2019/papers/Kirillov_Panoptic_Segmentation_CVPR_2019_paper.pdf).**

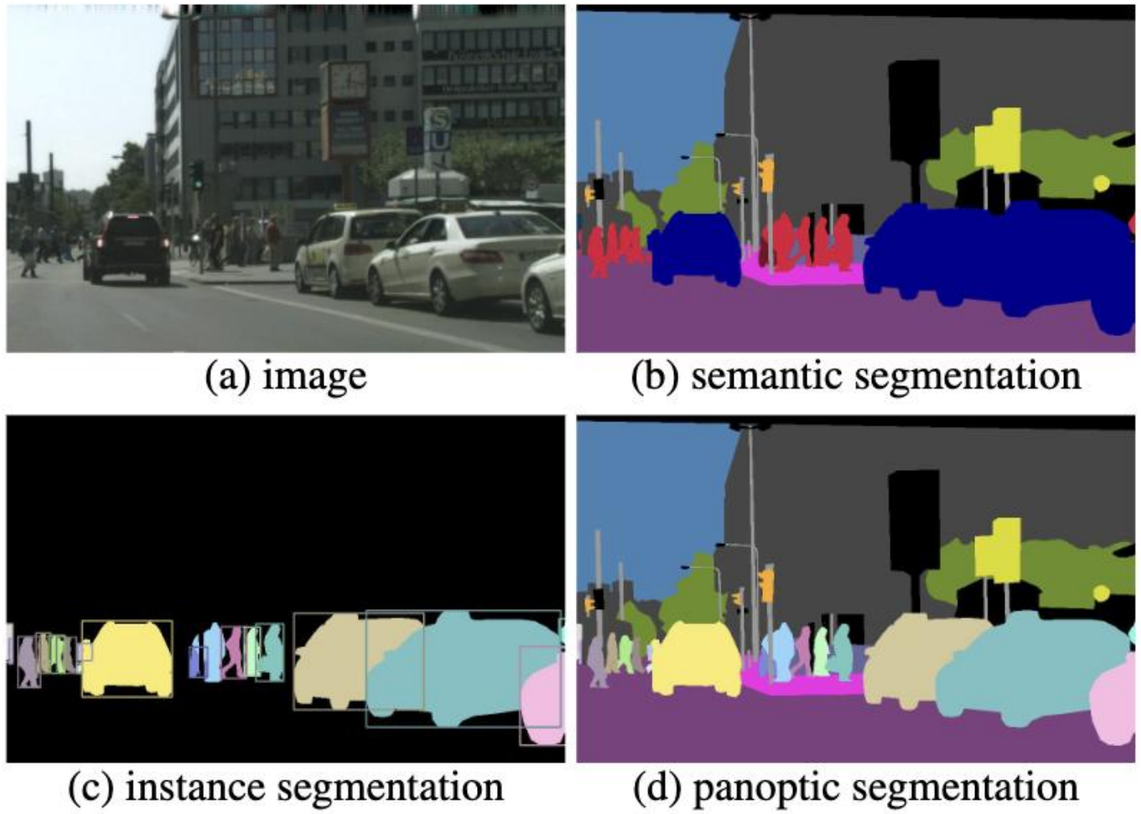
Object identification in images

In the image on the right we see the differences between Semantic, Instance, and Panoptic Segmentation.

We can see that the Panoptic Segmentation case—for the given image—is the best classification case because it is able to detect all entities.

In the case of Semantic Segmentation we see that all entities in the image are detected, but cars are not distinguished from each other nor people from each other.

In the case of Instance Segmentation, we see that cars and people can be distinguished from each other, but the algorithm is not able to detect uncountable objects, such as the road, the sky, and buildings.



Object identification in images

Given this, one might ask: once Panoptic Segmentation has been discovered recently, does it make sense to consider the other two methods?

As always in Engineering, there is no infallible method and it will always depend on the boundary conditions and the specific case we want to implement and the questions we want to answer.

Let's look at the differences between the main tasks from the mathematical statement of the methods:

- **Semantic Segmentation: assigns each individual pixel in the image a class-membership label. In this problem, the smallest unit of spatial resolution is the pixel.**

- **Instance Segmentation: assigns each entity (defined as a set of pixels) a label and a position. In this case, depending on the algorithm—and on how methods are classified in the literature—it is common to give the center of the object as the position and a square outline that encloses the object (bounding box), so all pixels inside the square outline are indistinguishable.**

Object identification in images

Additionally, from a cost–benefit perspective in real-world deployment, we find huge differences.

- **Semantic Segmentation: requires only a few labeled images to achieve good metrics and obtain an algorithm that delivers effective results. Although labeling all contours in a given image is laborious, with a few hundred images an algorithm can be trained.**

- **Instance Segmentation: requires a few thousand images for training. In addition, there is a very large difference in labeling complexity between labeling square bounding boxes and labeling the entire individual object contour, so if we need to detect all pixels of the surface, there will be a need to perform this task. Additionally, one must consider that Instance Segmentation-type models.**

- **Panoptic Segmentation: are very expensive in training time and also in inference time, so today combinations of Instance Segmentation + Semantic Segmentation are often used, which are more efficient.**

Object identification in images

Let's look at a problem with a dataset that, depending on what we want to do, requires one method or another.

In the image on the side, if our goal is “detect human constructions” and we want to detect them individually, we will use Instance Segmentation. If we use an algorithm that only detects using bounding boxes, the detection will be the yellow square.

If that is our task, and we don't need to go further, it is enough.

Object identification in images

But what if we want to distinguish the elements precisely?

In the attached image there are various construction elements: a house, a swimming pool, a paved patio… Also inside the bounding box there is a garden with grass that actually belongs to the neighbor, not to the property itself.

Would it be enough to train Instance Segmentation so that it only detects the house and not the rest of the elements? Would an algorithm that does not detect by bounding boxes and also detects pixels be sufficient?

Mathematical theory says yes, but reality is very complex, and getting an Instance Segmentation algorithm that works effectively in this context is very difficult. A Semantic Segmentation algorithm would do better.

Neighbor's property inside our bounding box

Object identification in images

Instance Segmentation algorithms work very well when it comes to detecting bounding boxes. However, to detect the elements inside the bounding box or to classify the content, they are usually combined with other algorithms.

For example, for pest detection in vineyard crops, Instance Segmentation can be used to detect the bounding box that contains the leaves. Afterwards, a standard CNN (ResNet, VGG...) can be used to classify whether the leaf is healthy or has a pest.

Usually, a simple CNN run on the bounding box from instance segmentation classifies better than the instance segmentation itself.

Neighbor's property inside our bounding box

Object identification in images

Within the field of object identification in images there are various topologies with different approaches, each with its pros and cons. In this topic we will study an example of each type:

- Classical methods: clustering and Chan–Vese
- Semantic Segmentation: U-Net.
- Instance Segmentation: YOLO.

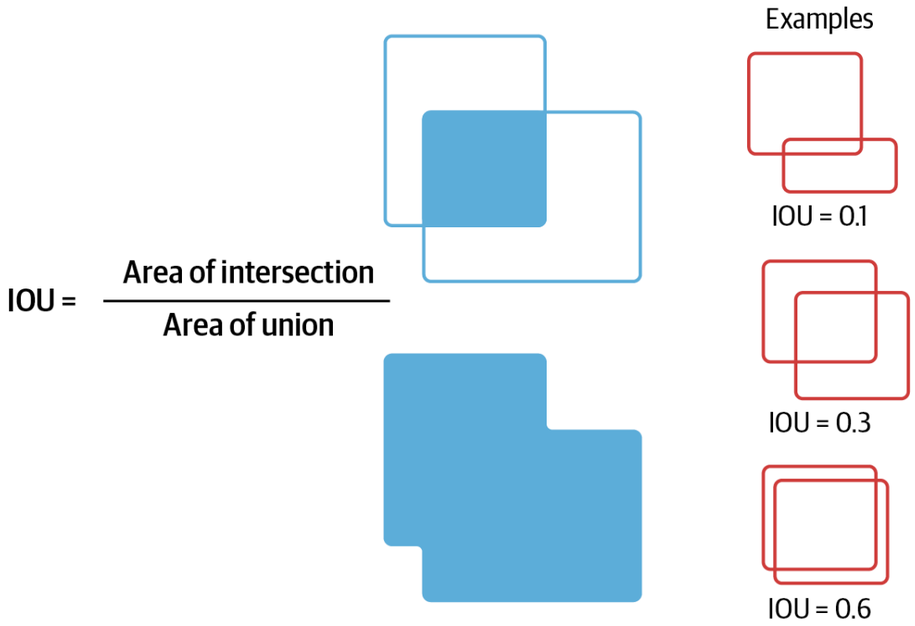
Evaluation Metrics

In the case of a classification algorithm, it is trivial to define metrics that indicate the model accuracy: it is enough to count how many records we classify correctly in a dataset with known labels.

In the case of Instance or Semantic Segmentation, this is not enough. We also need to know how much of the object area in the image we detect correctly.

For that we will use the Jaccard Index, also known as Intersection over Union (IoU).

$$IOU = \mathcal{J}(I_{pred}, I_{true}) = \frac{|I_{pred} \cap I_{true}|}{|I_{pred} \cup I_{true}|}$$



Evaluation Metrics

When evaluating Instance Segmentation, another metric that is often reported is mean Average Precision (mAP) defined over an IoU interval.

For example:

$$mAP_{[0.5-0.95]} = 0.5$$

This means we have an average precision of 0.5 for detecting objects with an IoU between 0.5 and 0.95.

Evaluation Metrics

Another aspect to take into account is determining the loss function.

To do so, we must keep in mind that Deep Learning consists of determining the values of weight tensors that define a neural network such that the error (or distance) between the network predictions and the real values of a training dataset is minimized. The function that describes that error is the loss function.

In the cases seen previously, where we classified an entire image into specific classes, the algorithm output was a rank-1 tensor with as many elements as classes to determine. The value of each element was the probability that the input image belonged to a given class. In that case, we used the NLL (negative log-likelihood) function.

$$\mathcal{L}_{NLL} = -\sum_{\mu} w_{\mu} \cdot y_{\mu}$$

Where w is the weight of the neural network output tensor, the index runs over the number of classes to predict, and y is the true value in the training dataset.

Evaluation Metrics

If what we want to predict are elements within an image and assign meaning to individual pixels or sets of pixels, the NLL loss function does not apply.

Moreover, the neural network output tensor will be something more generic and complex than a rank-1 tensor with as many elements as classes to predict.

Therefore, for Computer Vision tasks that go beyond simple image labeling, generic solutions will not be sufficient and we must examine each case. In particular, we must pay attention to:

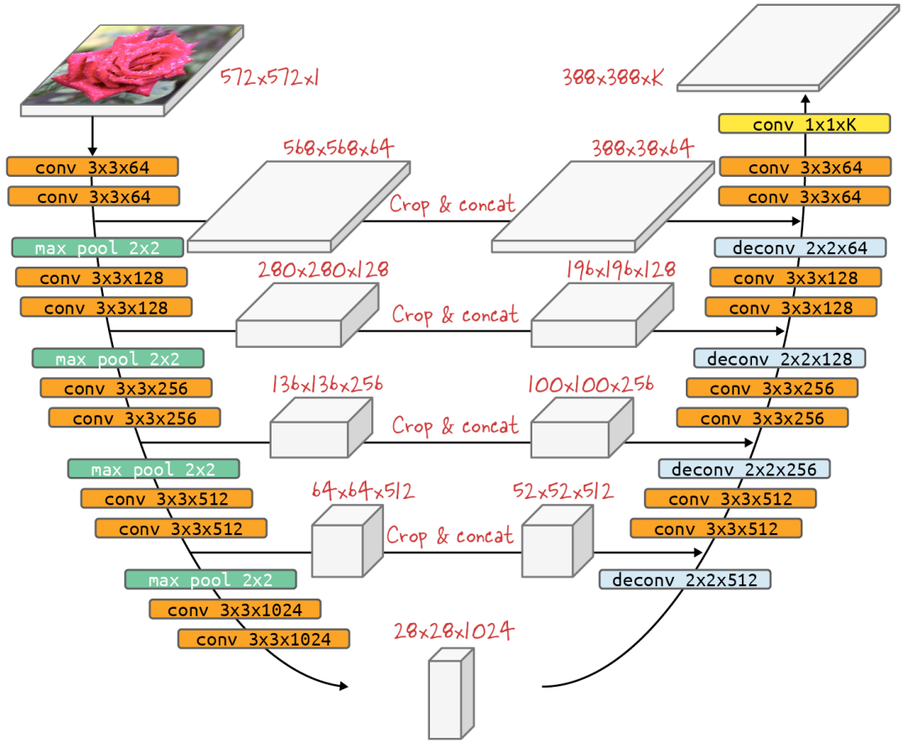
- **The rank of the neural network output tensors and what each element or tensor dimension means.**
- **The loss function to use, which will be determined by the output tensors.**

U-Net

U-Net is a very simple (2015) and effective topology for performing Semantic Segmentation tasks.

The strategy concatenates convolutional layers from the image down to a 28 x 28 pixel, 1024-channel feature pyramid. This feature pyramid is then deconvolved to reconstruct an image with the same size as the input image.

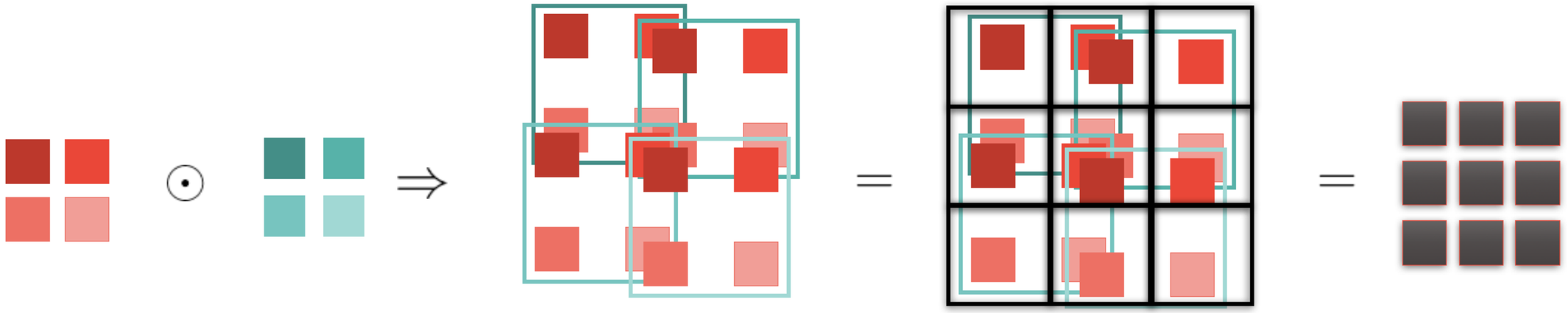
https://arxiv.org/abs/1505.04597



U-Net Topology

Transposed convolution works the opposite way to standard convolution. Starting from an image, it is multiplied by the transposed-convolution filters, but for each filter iteration the elements are not all summed (as in standard convolution).

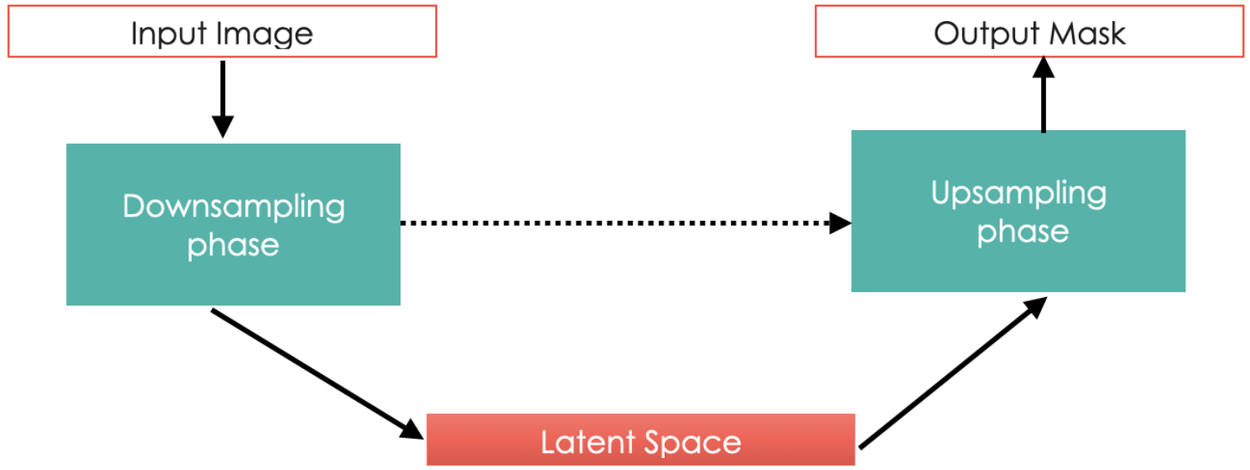
If, in two consecutive iterations of the filter over the image, there is overlap, the corresponding elements from all overlapping iterations are summed.



U-Net Topology

U-Net is made up of 3 clearly differentiated phases:

- The downsampling phase: it is a concatenation of several elementary blocks. The output of each downsampling block is connected to the input of the corresponding block in the upsampling phase.
- The latent space, where "textures" are represented. It is similar to the encoder–decoder formalism, but it is not exactly the same because there are connections between the "encoder" and the "decoder" layers.
- The upsampling phase, made up of elementary blocks, is connected to the downsampling phase.



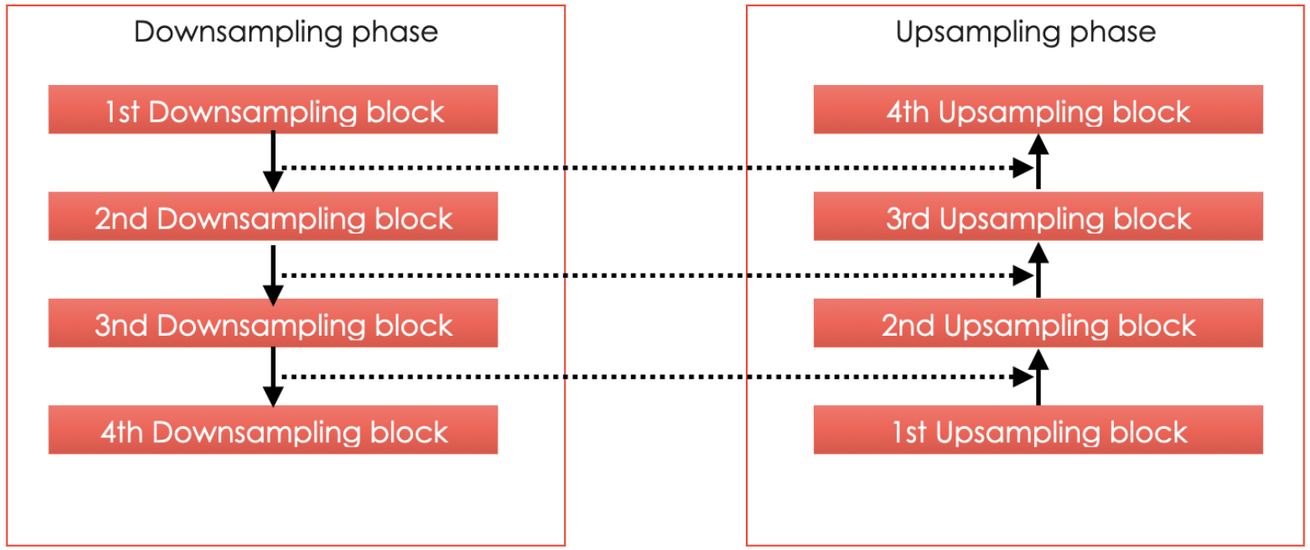
U-Net Topology

In U-Net, the output of each block in the downsampling phase is connected to the input of each block in the upsampling phase.

These connections are made in a very specific way:

- First downsampling – last upsampling.
- Second downsampling – second-to-last upsampling.
- Etc.

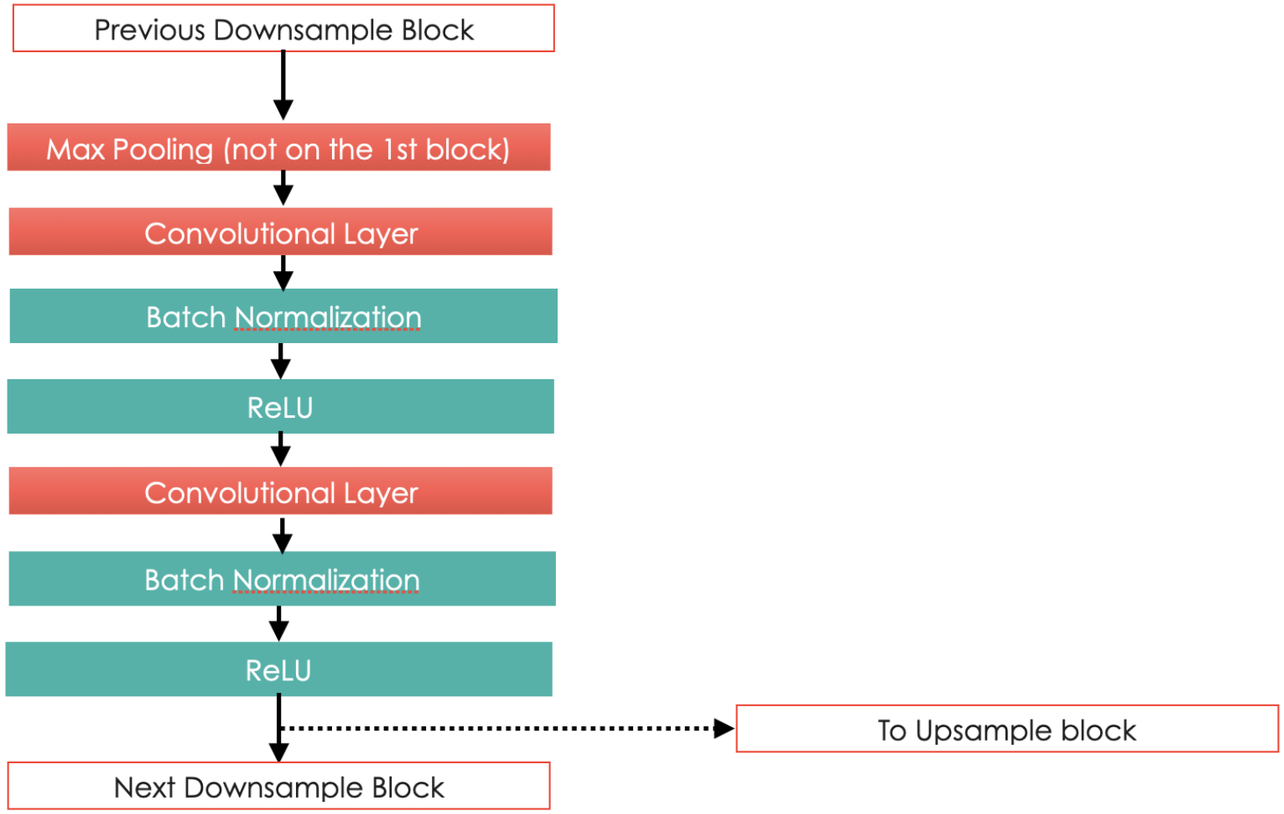
In this way, the connection is made such that the downsampling output is concatenated along the channel dimension with the input tensor of the upsampling phase.



U-Net Topology

Each Downsample block is composed of

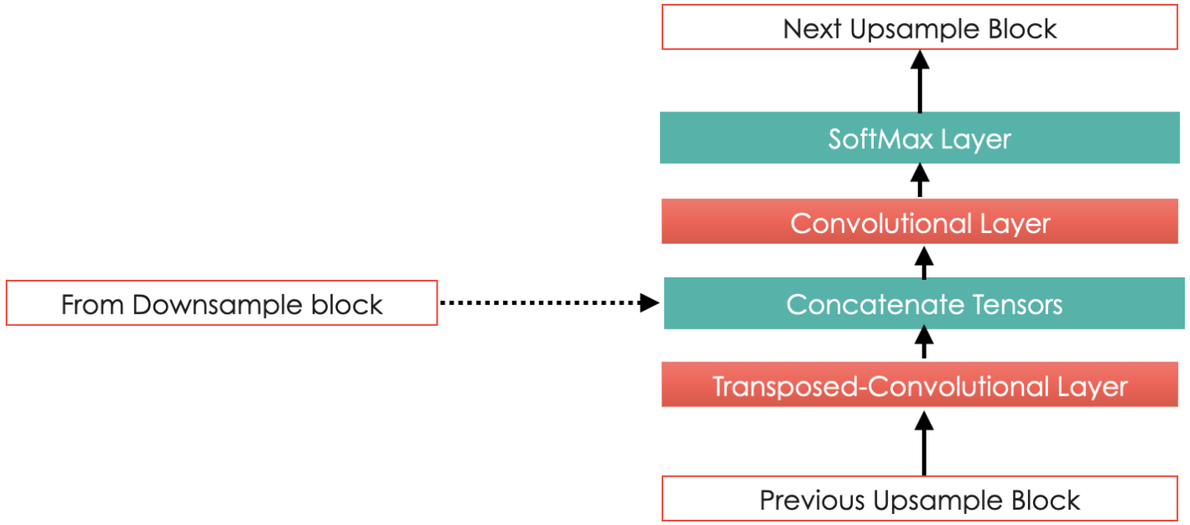
- A Max Pooling layer at the beginning. If it is the first Downsample block of U-Net, it receives the image directly.
- A concatenation of two convolutional layers, each followed by a Batch Norm stage and a ReLU activation layer.
- The output of each downsampling block is passed to the next block and also to the corresponding block in the upsampling phase.



U-Net Topology

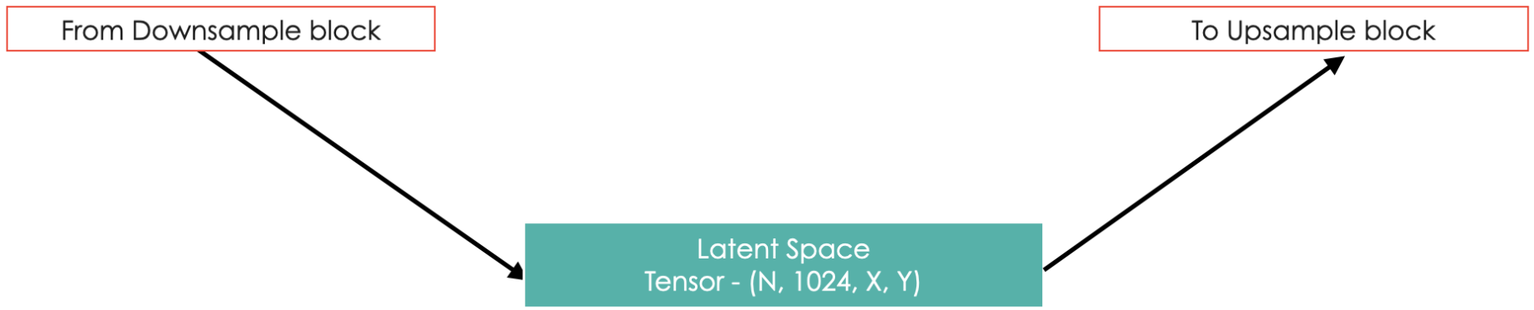
Each Downsample block is composed of:

- A Transposed Convolution layer that receives the output of the previous block in the Upsample phase. It can also
- The concatenation of the tensors received from the downsampling phase with the tensors output by the Transposed Convolution layer.
- A standard Convolution layer that takes as input the concatenation of the two previous tensors.
- A softmax activation layer.



U-Net Topology

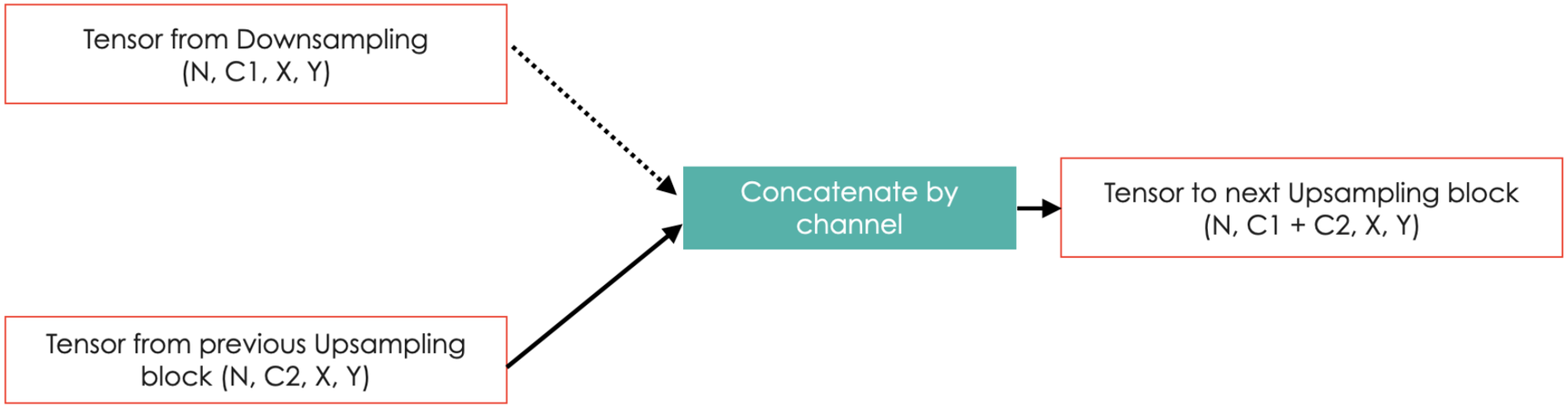
The latent space is defined by a single tensor and marks the separation between the downsampling and upsampling phases.



U-Net Topology

The concatenation takes as input the tensor from the downsampling phase and concatenates it with the tensor from the previous upsampling block.

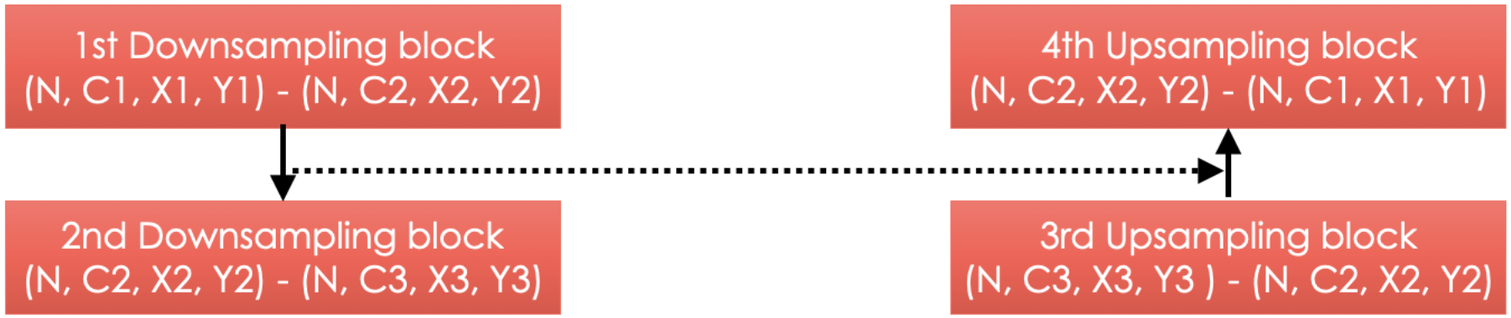
They are concatenated along the color-channel dimension. This implies they must have the same spatial resolution. If they do not, padding must be applied to the tensor with the smaller spatial resolution.



U-Net Topology

Concatenating tensors by channels imposes constraints on the input and output channels of convolutional and transposed-convolution layers.

The input and output channels of the downsampling stage are symmetric to those of the upsampling stage. That is, the input channels of the downsampling phase must match the output channels of the corresponding upsampling stage.



$$C_{n+1} + C_{n+2} = C_n \sim C_{n+1} = 2 \cdot C_n \iff C_n \in 2^n$$

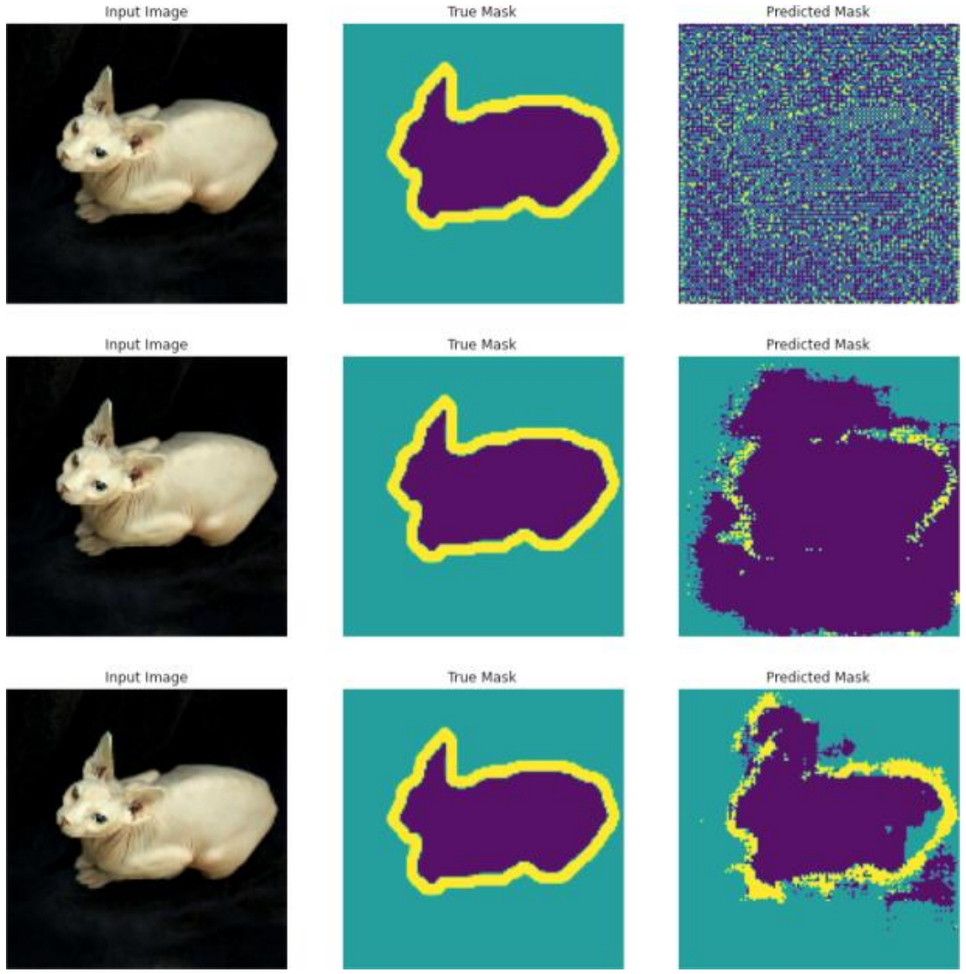
U-Net

The question now is: given a topology like the one above, how can we assign the correct class to each pixel?

To do so we must consider how to label the images. In the past, when we wanted to assign a global classification to an image, we assigned a single label to the image as a whole.

Now, we must assign a label to each individual pixel. This label indicates which class each pixel belongs to, and each pixel can belong to only one class.

This set of labels will form a new image of the same size as the original image and with a single channel, called a mask.



U-Net

The mask defined above will be the prediction output produced by our U-Net.

$$\text{U-Net} : f : \mathbb{R}^3 \rightarrow \mathbb{N}^2 \ / \ f(I_{[xyc]}) = M_{[xy]}$$

Where I is the tensor representing the image and M is the tensor representing the mask.

Therefore, once we know the shape of the output tensor, we only need to choose the loss function, which will be given by the Cross-Entropy function.

$$\mathcal{L}_{Cross-Entropy} = -\sum_{\mu}\sum_{x}\sum_{y} M_{xy}^{\mu} \log(p_{\mu}(f(I_{xy})))$$

Where μ runs over all labels in the ground-truth mask, and p is the probability (measured via a Softmax) of image I for label μ in the predicted mask.

Fortunately, all Deep Learning frameworks include Cross-Entropy, which returns its value for each pixel, so we only need to sum all the values of the rank-2 tensor.

U-Net

We iterate over the images and the masks

We read the PNG image, convert it to a Tensor, and resize it to the size expected by U-Net

```
129  if __name__ == '__main__':
130
131      images = os.listdir('./flood/data/Image/')
132      masks = os.listdir('./flood/data/Mask/')
133
134      print(len(images), len(masks))
135
136      image_tensor = list()
137      masks_tensor = list()
138      for image in images:
139          dd = PIL.Image.open(f'./flood/data/Image/{image}')
140          tt = torchvision.transforms.functional.pil_to_tensor(dd)
141          tt = torchvision.transforms.functional.resize(tt, (100, 100))
142
143          tt = tt[None, :, :, :]
144          tt = torch.tensor(tt, dtype=torch.float) / 255.
145
146          if tt.shape != (1, 3, 100, 100):
147              continue
148
149          image_tensor.append(tt)
```

We convert to float and divide by 255 to normalize the RGB image between 0 and 1.

We append to the list

U-Net

We read the mask and convert it to a Tensor

We replicate the single existing channel into 3 so we can resize it, and then we remove the 2 replicated channels

```
151     mask = image.replace('.jpg', '.png')
152     dd = PIL.Image.open(f'./flood/data/Mask/{mask}')
153     mm = torchvision.transforms.functional.pil_to_tensor(dd)
154     mm = mm.repeat(3, 1, 1)
155     mm = torchvision.transforms.functional.resize(mm, (100, 100))
156     mm = mm[:1, :, :]
157
158     mm = torch.tensor((mm > 0.).detach().numpy(), dtype=torch.long)
159     mm = torch.nn.functional.one_hot(mm)
160     mm = torch.permute(mm, (0, 3, 1, 2))
161     mm = torch.tensor(mm, dtype=torch.float)
162
163     masks_tensor.append(mm)
```

We append to the list

We convert the different colored regions of the mask into labels using a one-hot encoder.

We reorder the tensor dimensions and convert to float due to the requirements of the network topology.

U-Net

We concatenate into a single tensor

We instantiate U-Net with 3 channels (RGB) and 2 classes to predict

```
164
165   image_tensor = torch.cat(image_tensor)
166   print(image_tensor.shape)
167
168   masks_tensor = torch.cat(masks_tensor)
169   print(masks_tensor.shape)
170
171   unet = UNet(n_channels=3, n_classes=2)
172
173   dataloader_train_image = torch.utils.data.DataLoader(image_tensor, batch_size=64)
174   dataloader_train_target = torch.utils.data.DataLoader(masks_tensor, batch_size=64)
175
176   optim = torch.optim.Adam(unet.parameters(), lr=1e-3)
177   cross_entropy = torch.nn.CrossEntropyLoss()
```

We define the data loaders to iterate in batches.

We declare the optimizer.

This is almost irrelevant, but Adam is slightly better than SGD

We define the loss function

U-Net

```
179     loss_list = list()
180     jaccard_list = list()
181     for epoch in range(10):
182         running_loss = 0.
183         unet.train()
184
185         jaccard_epoch = list()
186         for image, target in zip(dataloader_train_image, dataloader_train_target):
187
188             pred = unet(image)
189
190             loss = cross_entropy(pred, target)
191             running_loss += loss.item()
192
193             loss.backward()
194             optim.step()
```

We iterate over epochs

We iterate in batches.

We set the network to train mode

We make a prediction

We compute the loss for this batch

We step the optimizer

We backpropagate the gradients

U-Net

```
195         for image, target in zip(dataloader_train_image, dataloader_train_target):
196
197             pred = unet(image)
198
199             _, pred_unflatten = torch.max(pred, dim=1)
200             _, target_unflatten = torch.max(target, dim=1)
201
202             intersection = torch.sum(pred_unflatten == target_unflatten, dim=(1, 2))/10000.
203
204             jaccard_epoch.append(torch.mean(intersection).detach())
205
206
207         jaccard_list.append(sum(jaccard_epoch)/len(jaccard_epoch))
208         loss_list.append(running_loss)
```

We iterate in batches to compute the metrics

We make a prediction

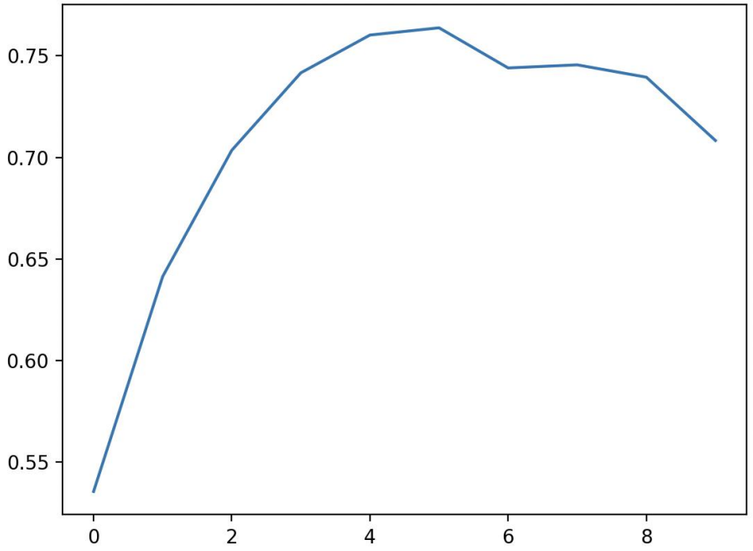
We evaluate the most probable class of the prediction for each individual pixel

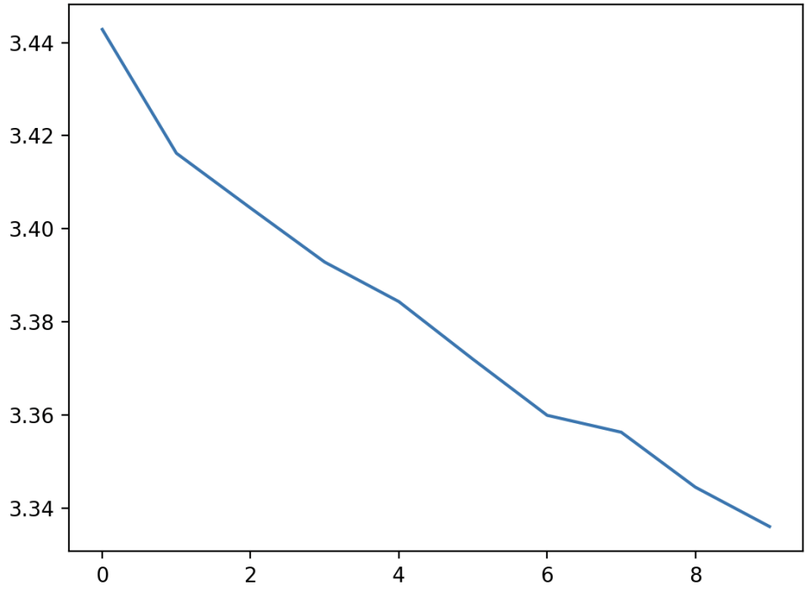
We evaluate the most probable class of the mask for each individual pixel

We compute the average intersection for each class and divide by the total image size (this is not the IoU).

U-Net

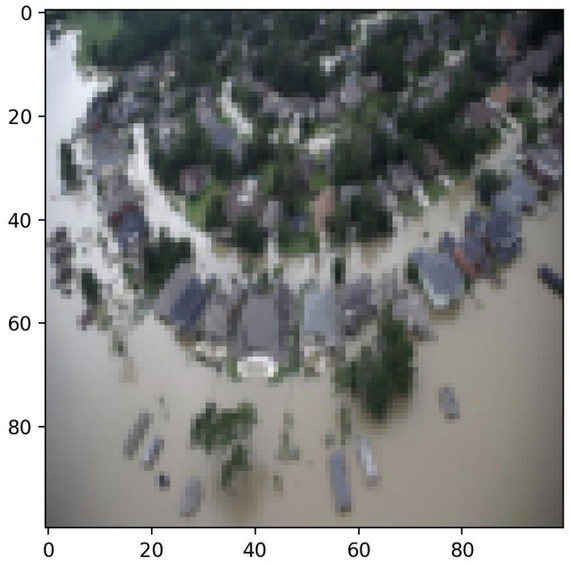
Below we see the IoU and the loss

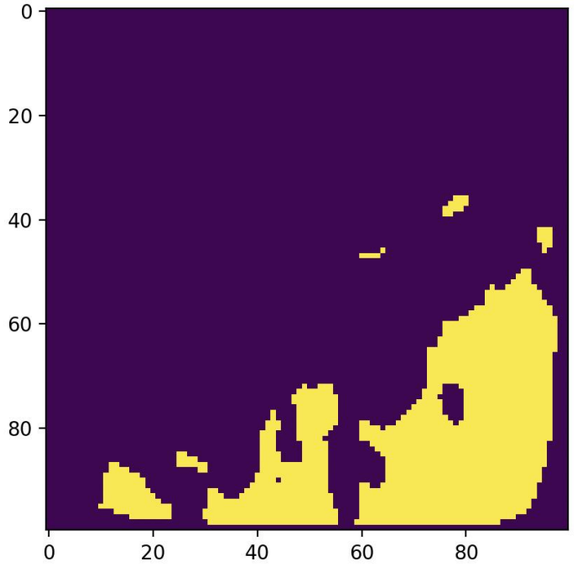


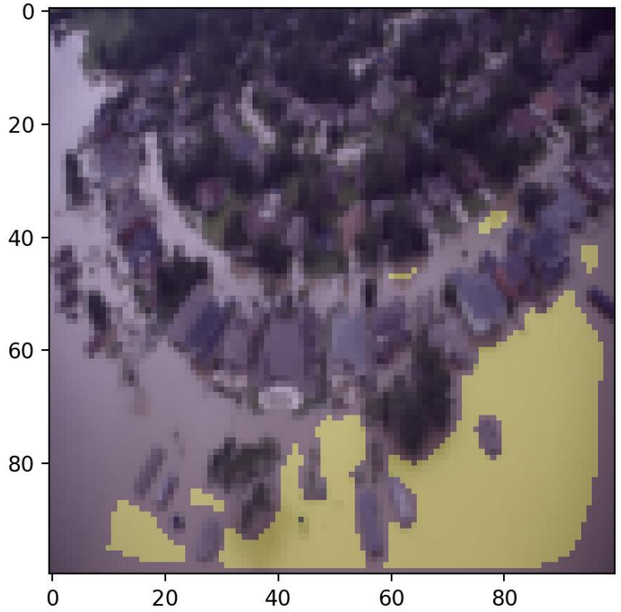


U-Net

Below we see the real image, the predicted mask, and both images overlaid.







Universidad Europea

manuel.garcia2@universidadeuropea.es

Dr. Manuel García Fernández

Ve más allá